

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
KATHMANDU ENGINEERING COLLEGE
KALIMATI, KATHMANDU



Major Project Report

On

KeyShard : Distributed Password Management System
[Code Number: CT 755]

BY

Ravi Bhusal	-	47213
Sachit Neupane	-	47215
Sushant Khanal	-	47237
Trishan Shrestha	-	47240

TO

DEPARTMENT OF COMPUTER ENGINEERING

Kathmandu, Nepal

Baishakh, 2083

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING

KATHMANDU ENGINEERING COLLEGE

KALIMATI, KATHMANDU



Major Project Report

On

KeyShard : Distributed Password Management System

[Code Number: CT 755]

PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF
COMPUTER ENGINEERING IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE BACHELOR OF ENGINEERING

BY:

Ravi Bhusal	-	47213
Sachit Neupane	-	47215
Sushant Khanal	-	47237
Trishan Shrestha	-	47240

Kathmandu, Nepal

Baishakh, 2083

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
KATHMANDU ENGINEERING COLLEGE
KALIMATI, KATHMANDU

CERTIFICATE

The undersigned certify that they have read and recommended to the Department of Computer Engineering a major project work entitled "KeyShard: Distributed Password Management System" submitted by Ravi Bhusal- 47213, Sachit Neupane- 47215, Sushant Khanal- 47237, and Trishan Shrestha- 47240 in partial fulfillment of the requirements for the degree of Bachelor of Engineering.

Er. Jalauddin Mansur
(Assistant Professor)
Department of Electronics and
Computer Engineering
Pulchowk Campus

Er. Anju Khanal
(Project Supervisor)
Department of Computer Engineering
Kathmandu Engineering College

Er. Sharad Chandra Joshi
(Project Coordinator)
Department of Computer Engineering
Kathmandu Engineering College

Er. Sudeep Shakya
(Head of Department)
Department of Computer Engineering
Kathmandu Engineering College

ACKNOWLEDGEMENT

This project is prepared in partial fulfillment of the requirement for the bachelor's degree in computer engineering. First and foremost, we offer our sincerest gratitude towards our college, Kathmandu Engineering College and Department of Computer Engineering, for providing us a platform to undertake this project. We express our gratitude to our Project Supervisor, **Er. Anju Khanal** for her valued suggestion and supervision. Our special thanks to the Head of Department, **Er. Sudeep Shakya**, the Deputy Head of Department, **Er. Kunjan Amatya** for their kind effort and help in guiding us in our project. We are thankful to our teachers, **Er. Mahesh Singh Kathayat** and **Er. Ritu Bajracharya** for providing valuable ideas and pointing out our improvement areas.

We owe our gratitude to our Year Coordinator and Major Project Coordinator, **Er. Sharad Chandra Joshi** for his motivation and continuous effort in providing us with valuable insights. We are extremely privileged to have got the opportunity to work on this project. Finally, we would like to thank our friends and families for their invaluable support.

Any kind of suggestion or criticism will be highly appreciated and acknowledged.

ABSTRACT

KeyShard is a distributed password management system designed to address the security limitations of centralized credential storage. The system distributes encrypted vault data across multiple BoltDB-backed nodes using consistent hashing and asynchronous replication, achieving a load distribution deviation of less than 0.3% from ideal uniformity. A zero-knowledge security model is enforced throughout, where the master password never leaves the client device and is processed locally using Argon2id. The resulting master key is then split via HKDF into cryptographically independent keys for authentication and vault encryption. All vault data is secured using AES-256-GCM authenticated encryption, and each vault modification requires a successful Ed25519 challenge-response verification before any change is applied. The backend is implemented in Go and deployed across five containerized nodes using Docker. For password strength assessment, an XGBoost classifier is trained on a balanced dataset of 600,000 real-world passwords, extracting fourteen structural and entropy-based features to achieve a macro-average accuracy of 86.55%. A security-adjusted probability threshold of 0.41 is applied to reduce false negatives on weak passwords, and SHAP values are computed per prediction to provide clear, feature-level feedback to users. The React Native mobile frontend runs the strength model on-device via ONNX Runtime, maintaining the zero-knowledge guarantee throughout.

Keywords: *Distributed Password Manager, Zero-Knowledge Security, XGBoost, AES-GCM, Argon2id, Machine Learning, Password Strength Analyzer, Docker, BoltDB.*

TABLE OF CONTENTS

ACKNOWLEDGEMENT.....	iv
ABSTRACT	v
TABLE OF CONTENTS.....	vi
LIST OF FIGURES	viii
LIST OF TABLES.....	ix
LIST OF ABBREVIATIONS	xi
1. INTRODUCTION.....	1
1.1 BACKGROUND THEORY	1
1.2 PROBLEM STATEMENT.....	4
1.3 OBJECTIVES	4
1.4 SCOPE	4
1.5 APPLICATION.....	5
2. LITERATURE REVIEW.....	6
2.1 DISTRIBUTED VS CENTRALIZED SYSTEMS.....	6
2.2 MACHINE LEARNING IN PASSWORD MANAGERS	7
2.3 AES-GCM Encryption in Zero-Knowledge Systems	8
2.4 EXISTING PROGRAMS AND THEIR WEAKNESSES.....	9
2.5 SOLUTION PROVIDED BY OUR SYSTEM.....	11
3. METHODOLOGY	13
3.1 PROCESS MODEL	13
3.2 SYSTEM BLOCK DIAGRAM	15
3.3 DATA FLOW DIAGRAMS.....	17
3.4 NECESSARY UML DIAGRAMS	19
3.5 FLOWCHART	22
3.6 ALGORITHM.....	23
3.7 TOOLS USED	28

3.8	SYSTEM DEVELOPMENT & IMPLEMENTATION	31
3.9	VERIFICATION AND TESTING	50
3.10	SYSTEM VALIDATION	66
4.	EPILOGUE	75
4.1	RESULT AND CONCLUSION.....	75
4.2	FUTURE ENHANCEMENTS	79
	REFERENCES.....	80
	BIBLIOGRAPHY	83
	SCREENSHOTS.....	84

LIST OF FIGURES

Figure 1.1.1	:	Distributed Systems	1
Figure 2.3	:	AES vs AES-CBC– Encryption and Decryption Time	8
Figure 3.1.1	:	Incremental Process Model	13
Figure 3.2	:	System Block Diagram	15
Figure 3.3.1	:	Data Flow Diagram (Level 0)	17
Figure 3.3.2	:	Data Flow Diagram (Level 1)	18
Figure 3.4.1	:	Use case Diagram	19
Figure 3.4.2	:	Sequence Diagram	20
Figure 3.4.3	:	Class Diagram of the system	21
Figure 3.5	:	System Flowchart	22
Figure 3.6.3	:	XGBoost Classifier	26

LIST OF TABLES

Table 3.9.1	:	Verification Strategy Matrix	51
Table 3.9.2.1	:	Argon2id Verification Results	53
Table 3.9.2.2	:	HKDF Key Separation Verification Results	54
Table 3.9.2.3	:	AES-GCM Verification Results	56
Table 3.9.3.1	:	Node Deployment Verification Results	57
Table 3.9.3.2	:	API Operation Verification Results	59
Table 3.9.3.3.1:		Key Distribution Across 5 Nodes (1000 keys)	60
Table 3.9.3.3.2:		Sharding Verification Results	60
Table 3.9.4.1.1:		Extracted Feature List and Value Ranges	61
Table 3.9.4.1.2:		Dataset Class Distribution	62
Table 3.9.4.2	:	Hyperparameter Search Space and Best Values	63
Table 3.9.4.3.1:		Classification Report on Test Set (Standard Threshold = 0.5)...63	
Table 3.9.4.3.2:		Classification Report on Test Set (Security-Adjusted Threshold = 0.41)	64
Table 3.9.4.3.3:		ROC-AUC Scores by Class	65
Table 3.9.4.4	:	Top 5 Features by Mean Absolute SHAP Value	65
Table 3.10.1.1	:	User Registration Validation	66
Table 3.10.1.2	:	User Authentication Validation	66
Table 3.10.1.3	:	Vault Operations Validation	68
Table 3.10.1.4	:	Session Management Validation	69
Table 3.10.1.5	:	Password Strength Analysis Validation	70
Table 3.10.2.1	:	Security Validation in Authentication	71
Table 3.10.2.2	:	Cryptographic Security Validation	72
Table 3.10.2.3	:	Security Validation on Input Handling	73

Table 3.10.2.4 :	Rate Limiting Validation	73
Table 3.10.3 :	System Performance Metrics	74

LIST OF ABBREVIATIONS

2FA :	Two-Factor Authentication
AEAD :	Authenticated Encryption with Associated Data
AES :	Advanced Encryption Standard
AES-GCM :	Advanced Encryption Standard – Galois/Counter Mode
AI :	Artificial Intelligence
AI/ML :	Artificial Intelligence / Machine Learning
API :	Application Programming Interface
BIP :	Bitcoin Improvement Proposal
CLI :	Command Line Interface
CTR :	Counter Mode (in cryptography)
CURL:	Client URL
DHT :	Distributed Hash Table
DiSPP :	Distributed Secure Password Protocol
GCM :	Galois/Counter Mode
GPU :	Graphics Processing Unit
HMAC:	Hash-based Message Authentication Code
HKDF:	HMAC-based Key Derivation Function
LSTM :	Long Short-Term Memory
MAC :	Message Authentication Code
ML :	Machine Learning
NLP :	Natural Language Processing
OAEP :	Optimal Asymmetric Encryption Padding
ORM :	Object-Relational Mapping
OWASP :	Open Web Application Security Project
RAFT :	Reliable, Replicated, Redundant, And Fault-Tolerant
REST :	Representational State Transfer
ROC :	Receiver Operating Characteristic
RSA :	Rivest–Shamir–Adleman
SHA :	Secure Hash Algorithm
SHAP:	Shapely Additive exPlanations
SVM :	Support Vector Machine
TF-IDF :	Term Frequency–Inverse Document Frequency

UI :	User Interface
VS Code :	Visual Studio Code
WSGI :	Web Server Gateway Interface
XGBoost :	Extreme Gradient Boosting
ZKPs :	Zero-Knowledge Proofs

1. INTRODUCTION

1.1 BACKGROUND THEORY

The exponential increase in the number of online services and digital accounts has led to a corresponding rise in the volume and complexity of user credentials. Passwords remain one of the most widely used forms of authentication. However, poor password practices such as reuse, weak formatting, and predictable structures have made users increasingly vulnerable to cyber-attacks. Traditional password management methods are no longer sufficient to protect against sophisticated attacks such as phishing, credential stuffing, and data breaches.

1.1.1 Distributed Management

Distributed system management refers to the process of overseeing and controlling the operation, configuration, and performance of distributed systems. It involves managing the various components, nodes, and resources that make up the distributed system to ensure its reliable, efficient, and secure operation. In decentralized management, decision-making authority, control, and coordination are distributed among multiple entities or nodes within the system. Each entity or node operates autonomously and has the power to make decisions independently within its scope of responsibility.

It includes logging and tracing, configuration management to ensure consistency, improve scalability, enhance reliability and facilitate change management, resource allocation to optimize performance, scheduling to minimize latency and maximize throughput along with fault detection and recovery[1].

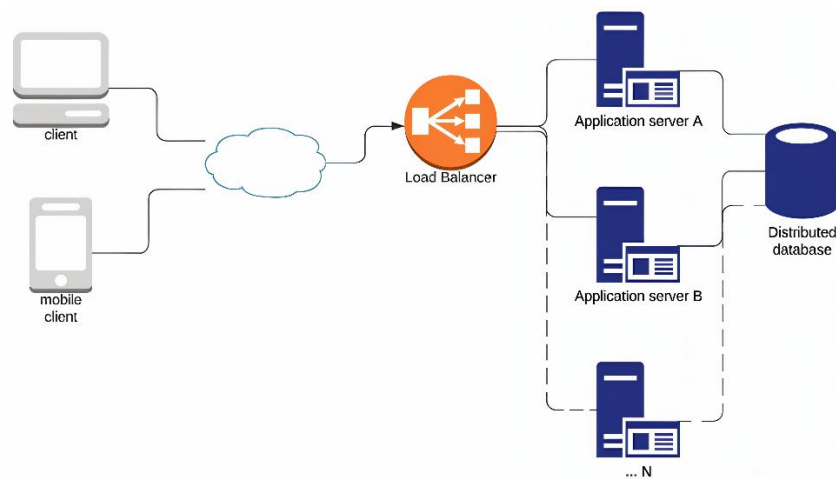


Figure 1.1.1: Distributed System

1.1.2 Password Managers

Password managers have emerged as essential tools that help users securely generate, store, and retrieve passwords. A password manager is a technology tool that helps internet users create, save, manage and use passwords across different online services. A password manager is an attempt to improve password usability and security, enabling users to create unique, complex passwords for every online account without needing to remember them. The username and password are then securely stored in the password manager. The next time the user visits the same site, the password manager opens a prompt window, typically above where the user input is required, asking if the user wants to input the previously saved information. On the other hand, when the user already has a username and password but visits a site for the first time with a password manager installed, it prompts the user to save account information for future visits.

Websites generally use a standard Hypertext Markup Language form for the username and password fields. Password manager technologies detect that username and password fields are present. The password manager also identifies the web address visited, matching it to a list of known credentials and determining if a password credential can be input or if a new password is needed. Browser developers and third-party password managers have different mechanisms for detecting username and password fields. Google has published a set of best practices to help developers build reliably detected usernames and password forms. Third-party password manager tools, including both 1Password and LastPass, have also published information to help developers build compatible forms[2].

1.1.3 Zero-Knowledge End-to-End Encryption

Zero-knowledge encryption ensures end-to-end data security and privacy within services like cloud storage and password management. Encryption is the process of converting information (data) into a special code to prevent unauthorized access to that information. Unencrypted messages are referred to as plaintext. Once encrypted, they become ciphertext. Encryption keys are used to encrypt or decrypt a message. Zero-knowledge encryption is not a specific encryption protocol, but a process that focuses on preserving a user's data privacy and security to the maximum extent. For a service to be truly zero-knowledge, a user's data must be encrypted before it leaves the device,

while it's being transferred, and when it is stored on an external server. The server can verify the user using Zero Knowledge Proofs[3].

Zero-knowledge proofs (ZKPs) are a cryptographic method used to prove knowledge about a piece of data, without revealing the data itself. At a high level, a zero-knowledge proof works by having the verifier ask the prover to perform a series of actions that can only be performed accurately if the prover knows the underlying information. The fundamental characteristics of ZKP are completeness, soundness and zero knowledge[4].

1.1.4 AES-GCM Based Authenticated Encryption

AES-GCM (Advanced Encryption Standard – Galois/Counter Mode) is an authenticated encryption algorithm that provides both data confidentiality and integrity in a single, efficient operation. It belongs to the class of AEAD (Authenticated Encryption with Associated Data) schemes and is widely adopted in modern security-critical systems [5].

AES-256-GCM is used exclusively for encrypting password vault data. Encryption keys are never transmitted over the network. Instead, they are derived locally on the client using a memory-hard key derivation function (Argon2id), followed by key separation using HKDF.

This design eliminates the need for asymmetric key exchange mechanisms such as RSA-OAEP, while still maintaining strong cryptographic guarantees under a zero-knowledge security model. The authentication tag generated by AES-GCM ensures detection of any unauthorized modification or tampering of encrypted vault data.

1.1.5 Argon2id Hashing Algorithm:

Argon2id is a modern, secure, and memory-hard password hashing algorithm, combining the benefits of Argon2i (resistant to side-channel attacks) and Argon2d (resistant to GPU cracking). It's the recommended variant for password hashing and key derivation. Argon2id is designed to protect against both side-channel attacks and highly parallelized cracking attacks by using a hybrid approach. The Argon2id function is recommended by OWASP for password hashing as it provides a balanced defense against both side-channel and GPU-based attacks[6].

1.1.6 ML-Driven Password Analysis

ML-driven analytics significantly enhances password security by using machine learning to identify weak passwords, detect unusual login patterns, and predict potential breaches. By analyzing vast datasets, ML can identify patterns in user behavior and potential vulnerabilities that human analysts may miss. ML systems are trained to understand not only what constitutes typical behavior but also the nuances of potential malicious activity.

Rule-based systems (e.g., zxcvbn) score passwords on length, character variety, and dictionary matches. However, these heuristics can be brittle and fail to generalize to novel patterns. Machine learning models (logistic regression, XGBoost, or neural networks) trained on features like entropy, n-gram distributions, and breach frequency can capture nuanced signals of weakness. Integrating breach data (HaveIBeenPwned) enables real-time detection of compromised credentials[7].

1.2 PROBLEM STATEMENT

Traditional password managers rely on centralized storage, risking data breaches, single points of failure, and limited scalability. The distributed password manager offers better availability and resilience but face challenges in secure synchronization, zero-knowledge encryption, and user-friendly deployment. Integrating ML can enhance security by providing real-time password strength analysis, but combining ML capabilities with strong privacy and performance in a distributed system remains a complex challenge. Additionally, ensuring seamless scalability and maintaining consistency across distributed nodes are critical for reliable user experience.

1.3 OBJECTIVES

To design and implement a secure distributed password management system that safely stores user passwords and provides real-time password strength evaluation using a zero-knowledge security model.

1.4 SCOPE

The password manager using distributed system and ML project encompasses the development of a secure and scalable solution capable of efficiently managing user credentials and sensitive data across multiple distributed nodes while ensuring high availability, fault tolerance, and data consistency. This involves the design,

construction, and testing of distributed database components, integration of ML-based password strength assessment and recommendation modules, as well as the implementation of robust encryption and authentication mechanisms to safeguard stored information. Additionally, node health monitoring through heartbeat mechanisms, data synchronization through asynchronous replication, and system resilience through automated container recovery are incorporated into the system to uphold performance, security, and reliability over time.

1.5APPLICATION

There are several applications for the project, KeyShard: Distributed Password Management System. Some of them are:

- **Personal Password Management:** Secure storage and management of user passwords for individual users.
- **Organizational Password Storage:** Safe handling of internal service and system passwords within small teams or organizations.
- **Secure Password Protection:** Use of encryption and distributed storage to reduce the risk of data compromise.
- **Password Strength Assistance:** Real-time evaluation of password strength to help users create stronger passwords.
- **Backup and Recovery:** Distributed storage ensures improved availability and minimizes the risk of complete data loss.

2. LITERATURE REVIEW

The exponential growth of digital services and online platforms has necessitated the development of robust password management systems to address the critical challenges of cybersecurity in the modern digital landscape. Password managers have emerged as essential tools for enhancing security by generating, storing, and managing complex passwords across multiple platforms. However, traditional password management systems face significant limitations in terms of security vulnerabilities, usability concerns, and the inability to dynamically assess password strength using advanced computational methods[8].

2.1 DISTRIBUTED VS CENTRALIZED SYSTEMS

The exponential growth of digital data and increasing security threats have necessitated a fundamental shift from traditional centralized password management systems to distributed architectures[8]. Centralized password managers, while offering simplicity and ease of management, present inherent vulnerabilities that make them attractive targets for attackers. A centralized system is characterized by a single point of control where all data and processing are managed by a central server or authority, commonly seen in traditional banking and client-server architectures[9].

The primary advantages of centralized systems include reliability, direct control, and simplified management. However, these benefits come with significant drawbacks including single points of failure, potential bottlenecks, and scalability challenges. When all passwords and credentials are stored in one central location, attackers focus their efforts on breaching that single target, as evidenced by numerous high-profile breaches affecting millions of users[10].

In contrast, distributed systems distribute control across multiple nodes, reducing reliance on a single point of authority. This architecture provides increased resilience, enhanced transparency, and improved fault tolerance. Distributed password managers eliminate the need to trust a central authority by distributing encrypted data across multiple nodes, ensuring that even if some nodes are compromised, the complete dataset remains secure.

The RockYou password leak, where over 32 million user passwords were exposed due to inadequate security and centralized storage, highlighted the severe risks associated

with centralized password management systems. In such architecture, a single breach can compromise millions of credentials, enabling large-scale offline dictionary attacks. To address these vulnerabilities, Distributed Single Password Protocols (DiSPP) have emerged as a more resilient solution. These systems generate high-entropy secrets that are independent of user passwords and leverage threshold cryptography to distribute the secrets across multiple storage providers. This distributed architecture significantly raises the attack cost, requiring an adversary to compromise several storage providers and login servers simultaneously, thus mitigating the single point of failure seen in centralized systems like RockYou[11].

Research has shown that distributed password managers provide superior data availability guarantees compared to their centralized counterparts. The fault-tolerant nature of distributed systems ensures continuous access to credentials even when network connectivity is compromised, making them particularly valuable for remote deployments[10].

2.2 MACHINE LEARNING IN PASSWORD MANAGERS

The XGBoost (Extreme Gradient Boosting) classifier has emerged as a highly effective machine learning algorithm for password strength prediction, demonstrating superior performance compared to traditional NLP-based approaches and other classical algorithms. This gradient-boosted decision tree method excels at handling structured password features while maintaining computational efficiency.

XGBoost analyzes password strength through:

- Structural characteristics: Length, character diversity, and positional patterns.
- Sequence complexity: Special character distribution and digit placement.
- Entropy calculations: Predictability of character combinations.

The algorithm's built-in regularization prevents overfitting, making it particularly effective for password datasets containing millions of entries. Unlike NLP-dependent methods, XGBoost requires minimal text preprocessing while still achieving high accuracy through its ensemble of decision trees.

Comparative studies reveal XGBoost's advantages:

- Achieved 94% accuracy in multi-class classification (weak/medium/strong)
- Outperformed SVM and Logistic Regression in ROC metrics (0.98 vs 0.92)

- Processed 6.6 million passwords with 98% precision on a 000webhost leak dataset[12].

Password managers utilizing XGBoost typically implement:

- TF-IDF vectorization for basic pattern recognition.
- Feature importance ranking to identify critical security factors.
- Adversarial training that improves classification accuracy by 20% against manipulated inputs.

To improve interpretability, Tree SHAP (SHapley Additive exPlanations) is integrated with XGBoost, enabling feature-level explanations for each prediction. This allows the system to show users which factors such as length, character diversity, or sequence patterns most influenced their password's strength score, thereby increasing transparency and trust in automated evaluations.

2.3 AES-GCM Encryption in Zero-Knowledge Systems

AES-GCM (Advanced Encryption Standard in Galois/Counter Mode) alone is sufficient for securing sensitive data when encryption keys are derived and managed entirely on the client side. In KeyShard, encryption keys are generated locally using the Argon2id key derivation function from the user's master password, ensuring that keys never leave the client device. This client-derived key model follows a zero-knowledge security architecture, where servers only store encrypted data and associated metadata (nonce and authentication tag) and never have access to plain text or encryption keys. Under this design, asymmetric encryption for key exchange is unnecessary because there is no need to transmit or share encryption keys with the server.

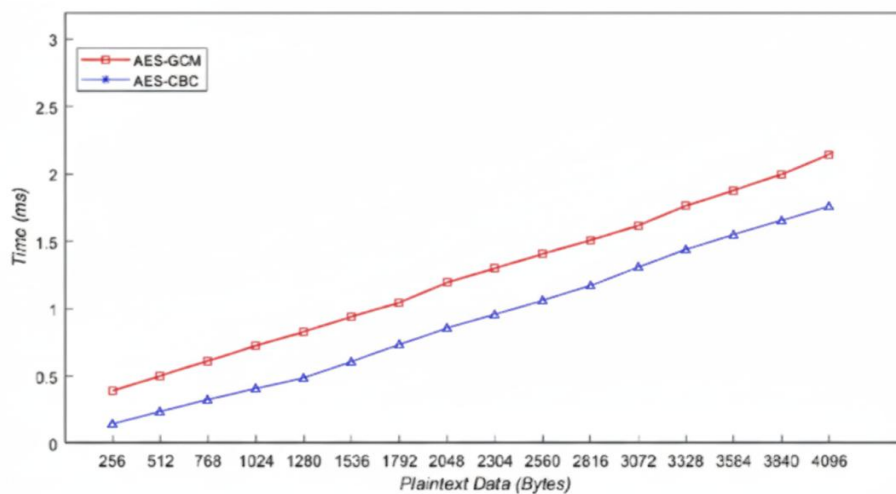


Figure 2.3 : AES vs AES-CBC– Encryption and Decryption Time

Modern password managers such as Bitwarden and 1Password intentionally avoid RSA-based hybrid encryption for vault protection for this reason. Instead, they rely on strong symmetric encryption (AES-256-GCM) combined with memory-hard key derivation functions like Argon2 or PBKDF2. This approach significantly reduces system complexity, eliminates asymmetric key management overhead, and improves performance on mobile and resource-constrained devices while maintaining strong confidentiality and integrity guarantees. AES-GCM, as an authenticated encryption scheme (AEAD), simultaneously provides encryption and tamper detection, making it well-suited for password vaults where data is encrypted before transmission and storage. Consequently, AES-GCM with client-side key derivation offers security equivalent to or stronger than hybrid encryption models in zero-knowledge password management systems, without the added computational and architectural cost of RSA.

2.4 EXISTING PROGRAMS AND THEIR WEAKNESSES

Contemporary password managers, while widely adopted, suffer from structural vulnerabilities inherent to centralized architectures and outdated password evaluation methods.

2.4.1 Centralized Architecture: A Single Point of Failure

Centralized password managers consolidate all user credentials and encryption keys within a single server or cloud infrastructure . This design creates a high-value target for attackers, as breaching the central repository grants access to all stored data .

Master Password Vulnerability, a compromised master password (e.g., via phishing or credential stuffing) provides attackers with full access to the vault[13].

In 2022, LastPass reported a breach where hackers exfiltrated encrypted vaults and targeted master passwords through brute-force attacks. The attacker leveraged valid credentials stolen from one of four senior DevOps engineers to access a shared cloud-storage environment[14].

There is risk of memory-based exploits, as Centralized systems often retain encrypted credentials in RAM for usability . Independent Security Evaluators demonstrated that 1Password, Dashlane, KeePass and LastPass stored master passwords in plaintext in memory, enabling extraction via cold-boot attacks. Scalability and Dependency: The centralized model also introduces scalability bottlenecks and dependency on third-party

providers. LastPass users experienced service disruptions due to centralized infrastructure failures, demonstrating the inherent risks of single-point dependency[15].

2.4.2 Lack of AI-Driven Password Analysis

Contemporary password managers suffer from a fundamental weakness in their approach to password strength evaluation, relying predominantly on outdated rule-based heuristics rather than advanced artificial intelligence and machine learning techniques. This limitation represents a critical gap in modern cybersecurity infrastructure, as traditional methods fail to account for the sophisticated attack vectors and linguistic patterns that characterize contemporary password cracking attempts.

2.4.2.1 Traditional Rule-Based Limitations

Existing password managers employ simplistic rule-based systems that evaluate password strength through predetermined criteria such as minimum length requirements, character diversity mandates, and the presence of uppercase letters, numbers, and special characters. These rule-based approaches suffer from significant variability across different organizations, where what constitutes a "strong" password in one system may be considered weak in another, creating inconsistent security standards across platforms[16].

The fundamental problem with rule-based systems lies in their inability to adapt to evolving security threats without manual updates, making them increasingly obsolete against modern attack methodologies[16]. These systems often fail to provide accurate assessments of password strength in real-world scenarios, particularly when passwords contain dictionary words, personal information, or predictable patterns that meet complexity requirements but remain highly vulnerable to targeted attacks[17].

2.4.2.2 Machine Learning Integration Gap

Password managers have largely ignored the potential of AI-driven password evaluation. Machine learning algorithms trained on large datasets of real-world passwords can identify patterns and characteristics associated with weak and strong passwords far more effectively than rule-based systems[16].

The benefits of machine learning integration include universality across different organizational policies, adaptability to emerging security threats without manual updates, and significantly improved accuracy in password strength assessment[16].

Neural networks, particularly Long Short-Term Memory (LSTM) networks, have demonstrated superior capability in identifying complex password structures and their resistance to brute-force attacks compared to traditional evaluation methods[18].

2.4.3 Distributed Database Systems in Password Managers:

While many password managers traditionally use centralized storage, a few modern solutions now incorporate distributed or decentralized architectures to improve security and availability. However, these approaches still come with their own limitations, such as scalability challenges, complex recovery, and limited security features, highlighting the need for a more advanced and balanced solution.

- 1. Vaultwarden:** Vaultwarden is an open-source, self-hosted alternative to the popular password manager Bitwarden. It's written in Rust and aims to provide a secure, and customizable solution for password management[19]. It is not officially supported and may face potential security risks since it is self-hosted. It has scalability constraints with backend limitations and relies on community contributions for bugfixes and feature updates.
- 2. Web3Password:** Web3Password stores encrypted credentials using BIP39/BIP44 mnemonic phrases (familiar from Bitcoin wallets), eliminating the need for a traditional master password[20]. Its major downside is that it is dependent on Web3 wallet with no 2FA with absence of AI/ML features. In it, storing encrypted data on a public chain exposes encrypted passwords indefinitely. If encryption weakens in future, attackers could brute force past entries. Similarly, if users lose their mnemonic phrase, they risk losing all data permanently as there is no recovery system.

Both Vaultwarden and Web3Password offer unique approaches to password management, but each has notable limitations. This highlights the need and opportunity for a more robust, secure, and user-friendly password manager that combines decentralization, strong recovery options, and advanced security features.

2.5 SOLUTION PROVIDED BY OUR SYSTEM

The “KeyShard: Distributed Password Management System” provides effective solutions to the security, scalability, and usability challenges associated with traditional password management systems. By integrating distributed system architecture, strong

cryptographic mechanisms, and artificial intelligence, the system delivers the following solutions:

1. Distributed Password Storage:

Encrypted password data is stored across multiple nodes, reducing the risk associated with centralized storage and improving fault tolerance.

2. Zero-Knowledge Security Model:

All encryption and key derivation processes are performed on the client side, ensuring that plain text passwords are never accessible to the server.

3. Strong Cryptographic Protection:

The system uses AES-GCM for authenticated encryption and Argon2id for password-based key derivation, ensuring confidentiality and integrity of stored data.

4. Machine Learning–Based Password Evaluation:

A trained machine learning model evaluates password strength in real time, helping users create stronger and more secure passwords.

5. Containerized Deployment:

Docker is used to simplify deployment, ensure consistent system behavior, and improve portability across different environments.

3. METHODOLOGY

3.1 PROCESS MODEL

3.1.1 INCREMENTAL MODEL

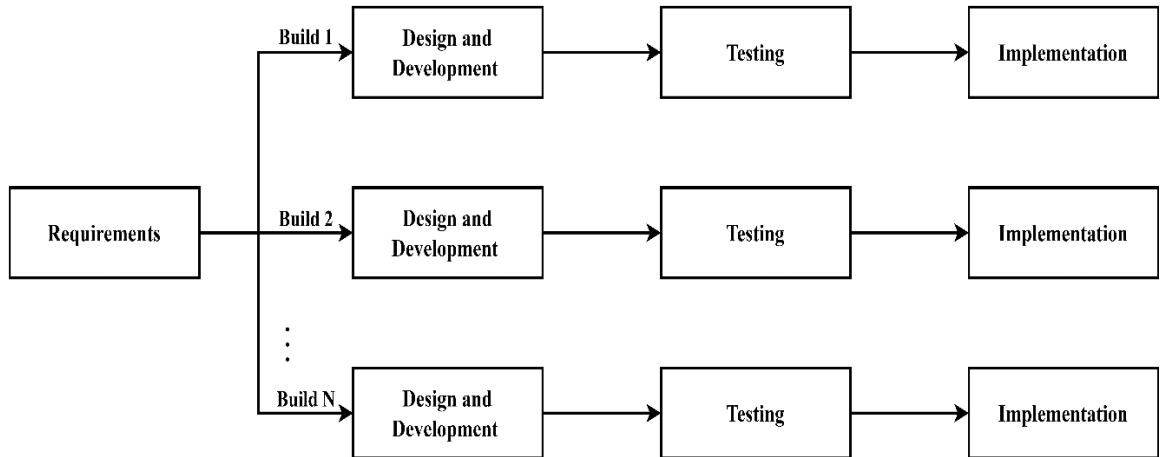


Figure 3.1.1 : Incremental Process Model

The Incremental Model is a process of software development where the overall system requirements are divided into multiple modules. Each module passes through the phases of requirements analysis, design and development, testing, deployment, and implementation. Every subsequent build adds functionality to the previous one, and the process continues until the complete password manager system is developed.

Requirement Analysis: Identification of core system needs like secure vault creation, ML-based password strength checking, distributed database management, zero-knowledge encryption, and data consistency. Password datasets, cryptographic algorithms and implementation for encryption/decryption process are analyzed to identify the requirement for the project. Analysis of the hardware requirements like RAM, OS, etc. for the device to maintain consistency among the devices. Also, deployment requirements are analyzed to define the scope and technical direction of each increment.

Design and Development: Designing modular architecture where each increment builds a key feature, such as distributed storage (using consistent hashing and BoltDB), ML-based password analysis using XGBoost, and authenticated vault encryption using

AES-256-GCM. Containerization using Docker supports consistent and portable deployment across all nodes.

Testing: Unit, integration, ML validation, and security tests are performed for each increment. Any bugs or vulnerabilities identified are resolved before proceeding to implementation.

Implementation: Each tested increment is deployed and integrated into the growing system. Continuous integration ensures smooth feature addition and system updates, with each increment building on the previous until the full system is complete.

3.2 SYSTEM BLOCK DIAGRAM

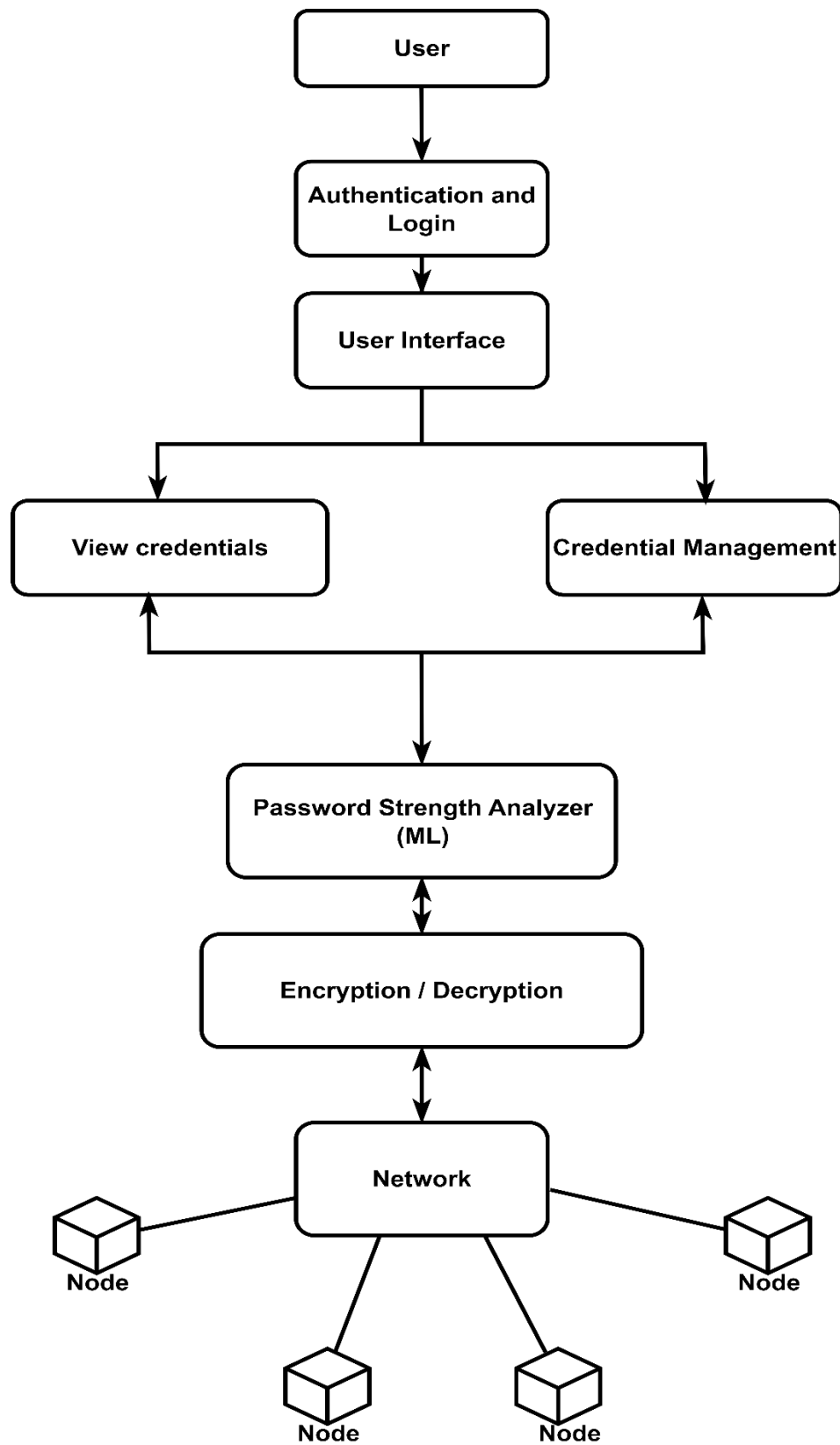


Figure 3.2 : System Block Diagram

The system block diagram of the “KeyShard: Distributed Password Management System” represents a secure and intelligent architecture designed for robust password management using machine learning and distributed storage principles.

The process begins with the User, who interacts with the system via a user-friendly interface. Access is controlled through a secure Authentication and Login module, which ensures that only verified users can access the vault. This may include standard credentials like usernames and passwords, along with optional multi-factor authentication (2FA) or biometric login methods.

Once authenticated, the user accesses the User Interface, which allows them to view, add, update, or delete credentials. When a new password is created or modified, it is passed through the Password Strength Analyzer, a machine learning module trained to assess password robustness. This component evaluates the password against patterns from known data breaches or entropy models and provides feedback or suggestions for stronger alternatives.

Before storing, all credential data is passed through the Encryption/Decryption module, which applies advanced cryptographic techniques (e.g., AES-256-GCM encryption with client-side key derivation using Argon2id). This ensures that all user data remains protected both in transit and at rest.

The Network layer facilitates secure communication between the application and the distributed storage backend. It also manages the coordination and synchronization of Nodes, individual distributed units responsible for storing encrypted password data. These nodes are designed to maintain high availability and resilience, even in the event of node failures. They implement distributed consensus protocols such as RAFT (for strong consistency) or Gossip protocols (for high scalability and eventual consistency), depending on the storage model.

Together, this architecture offers a scalable, intelligent, and highly secure password management system that combines the strengths of machine learning and distributed systems for modern cybersecurity needs.

3.3 DATA FLOW DIAGRAMS

3.3.1 DFD LEVEL 0

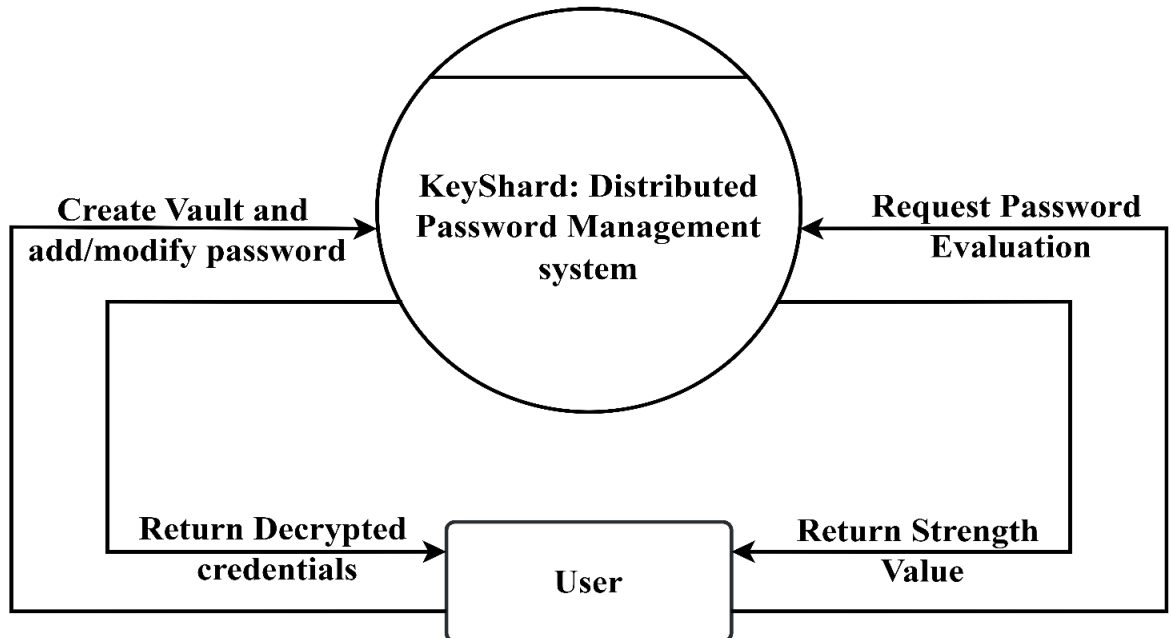


Figure 3.3.1 : Data Flow Diagram (Level 0)

The Data Flow Diagram level 0 (DFD 0) represents the overall system as a single process interacting with the user. The user provides inputs such as registration details, login credentials, and password-related requests to the system. The system processes these inputs and returns responses such as authentication status and credential access results.

3.3.2 DFD LEVEL 1

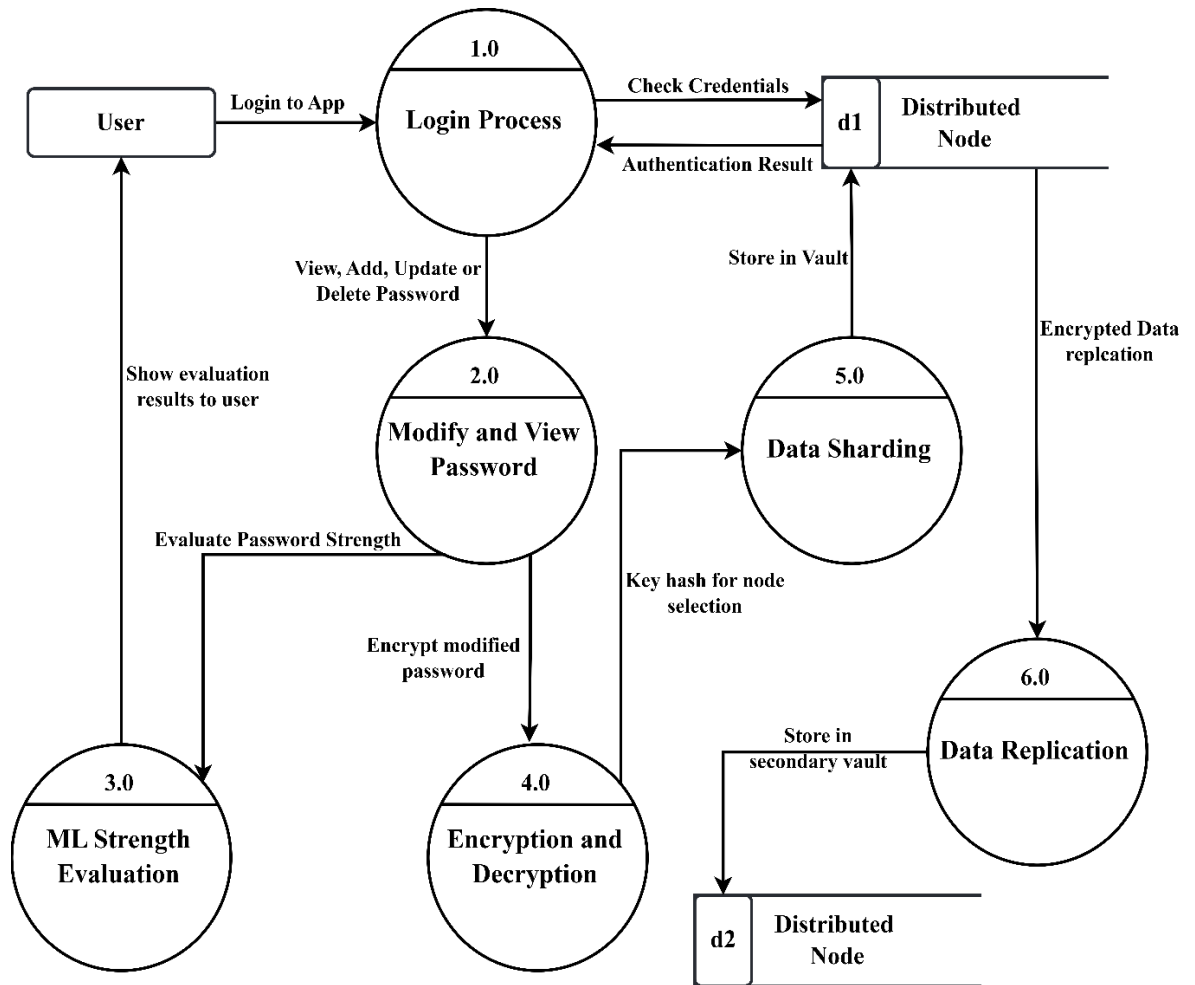


Figure 3.3.2 : Data Flow Diagram (Level 1)

This diagram expands the system into multiple internal processes showing how data flows between them. User inputs are first handled by the authentication process, after which credential-related requests are processed through encryption and secure handling modules. Encrypted data is stored and retrieved from internal data stores. The diagram clearly illustrates the flow of sensitive information through defined processes before reaching storage or being returned to the user.

3.4 NECESSARY UML DIAGRAMS

3.4.1 USECASE DIAGRAM

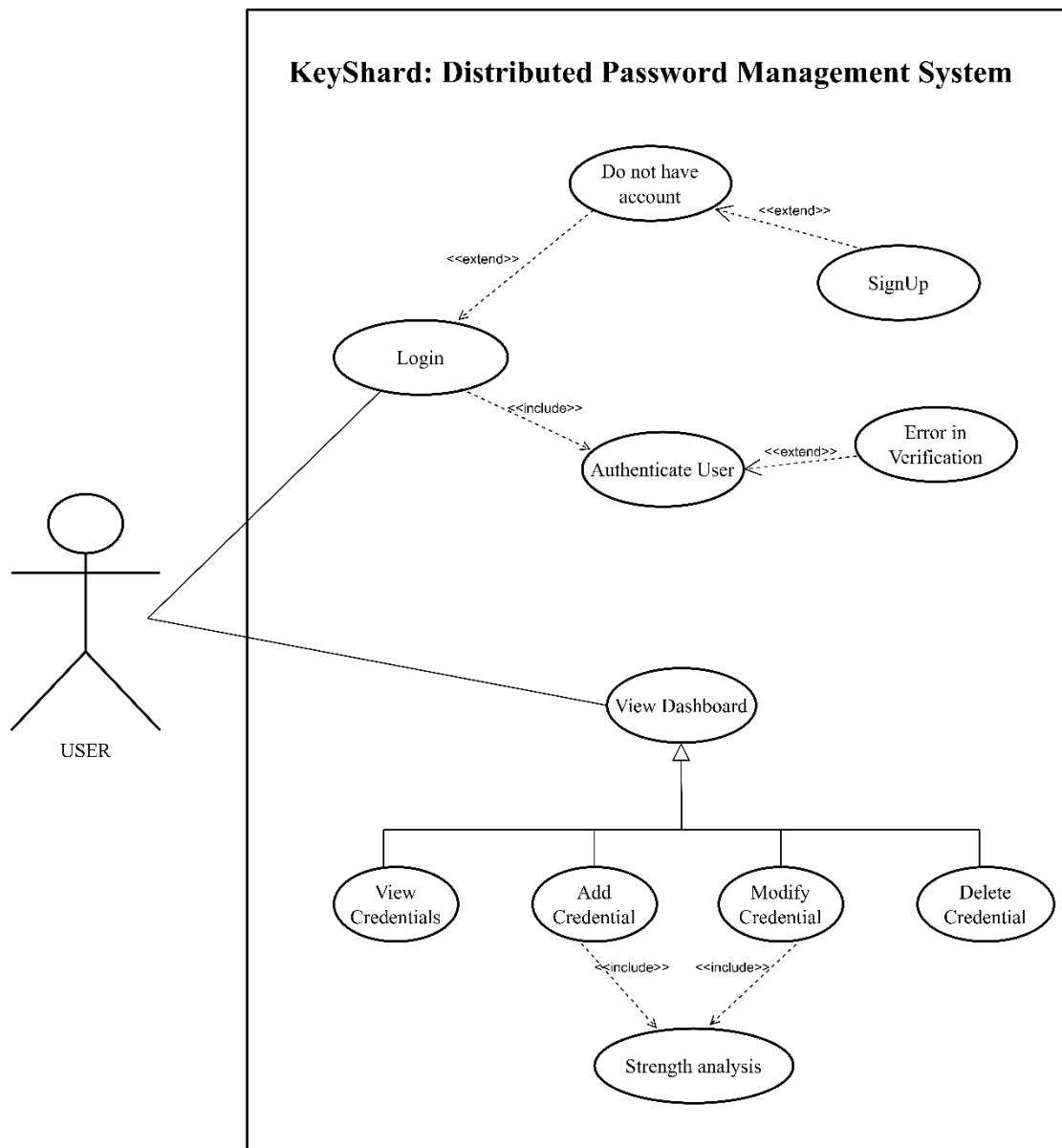


Figure 3.4.1 : Use case Diagram

This use case diagram illustrates the functional interactions between the user and the system. The user can perform actions such as account registration, login, storing credentials, retrieving stored credentials, and managing existing data. Each use case represents a specific function provided by the system and highlights that all operations are initiated by the user.

3.4.2 SEQUENCE DIAGRAM

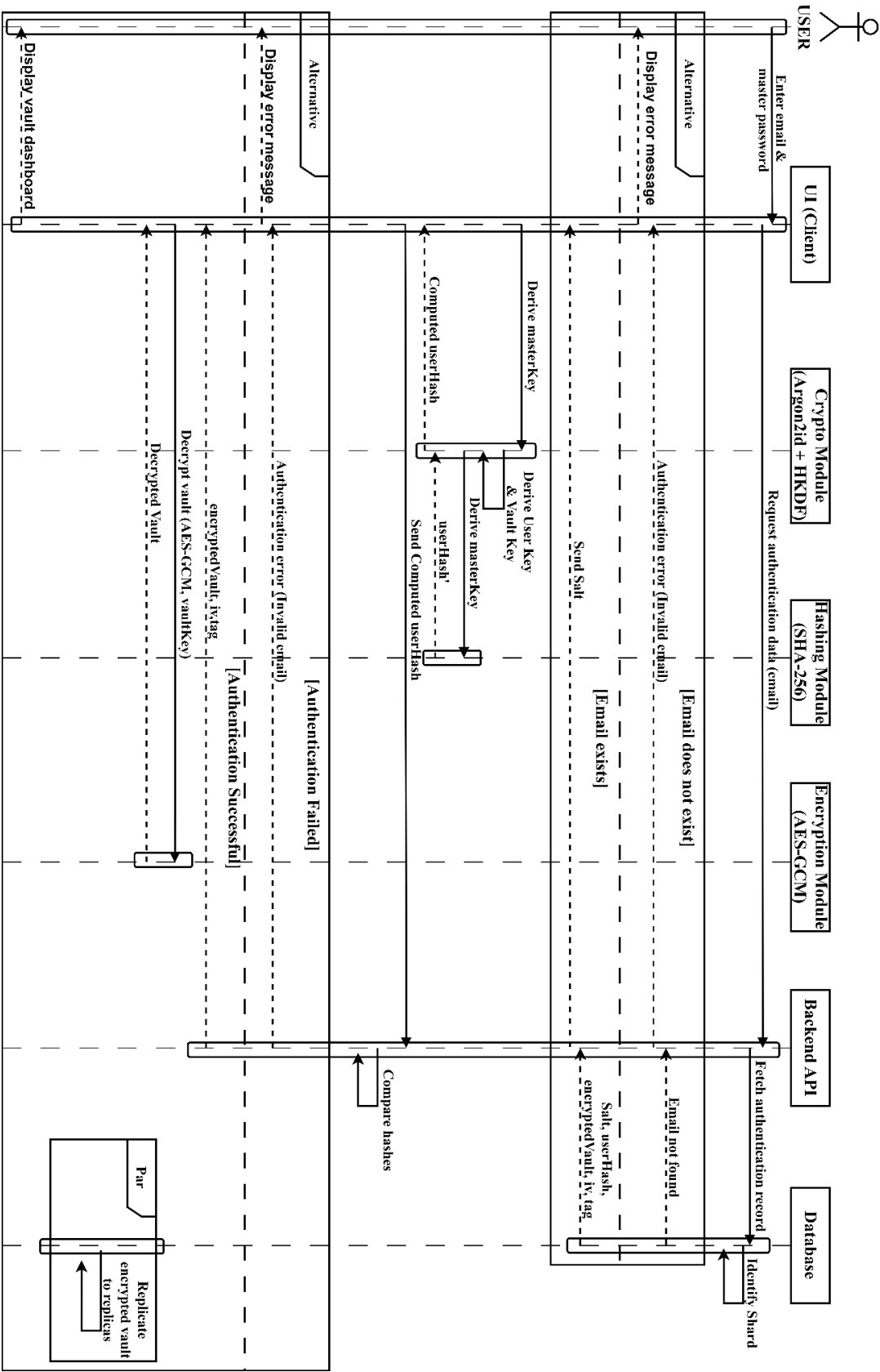


Figure 3.4.2 : Sequence Diagram

This sequence diagram illustrates the interaction flow between the user, application interface, security modules, machine learning component, and distributed storage system. It shows how user authentication, password strength evaluation, encryption, and distributed storage are executed in a secure and sequential manner, ensuring data confidentiality and system reliability.

3.4.3 CLASS DIAGRAM

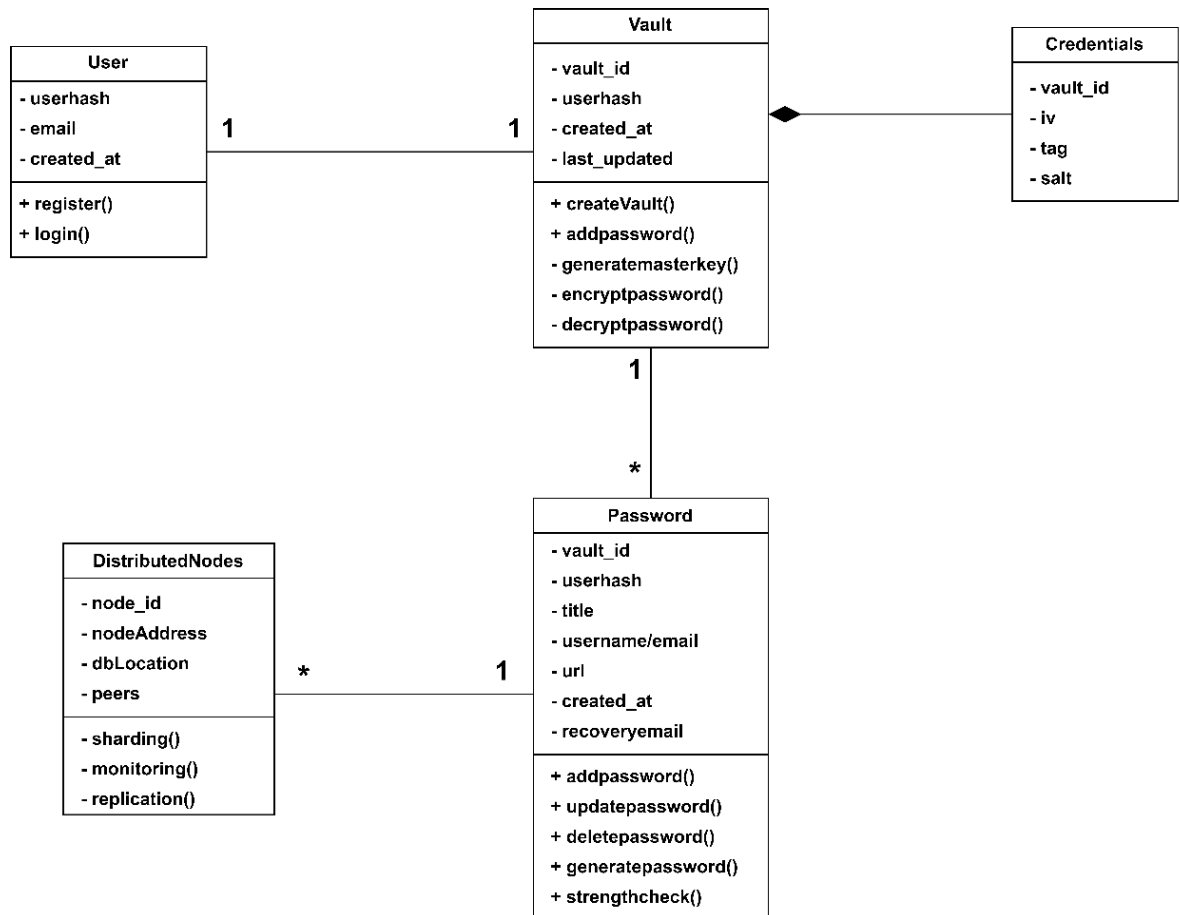


Figure 3.4.3 : Class Diagram of the system

The class diagram represents the overall system architecture of the KeyShard application. It shows the key classes and their relationships, including user authentication, password management, encryption services, machine learning–based password analysis, and distributed storage components. This diagram highlights the modular and scalable design of the system.

3.5 FLOWCHART

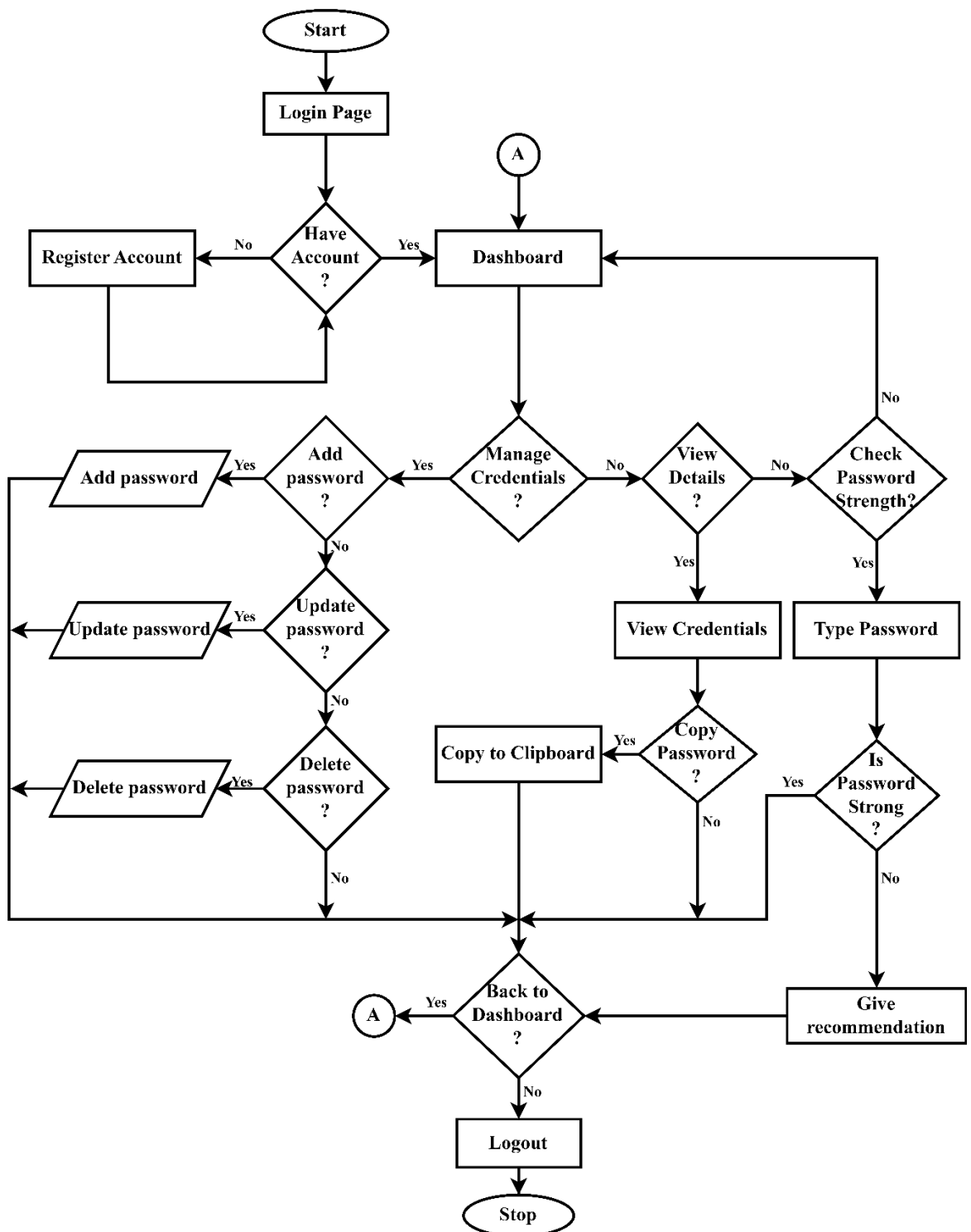


Figure 3.5 : System Flowchart

3.6 ALGORITHM

3.6.1 AES-GCM Vault Encryption Algorithm

Encryption Steps:

Input:

- data – Plaintext vault data (string or JSON object)
- key – 256-bit Vault Key derived on the client (Argon2id + HKDF)

Output:

- encryptedVault – Ciphertext
- iv – Initialization Vector (nonce)
- tag – Authentication tag

Steps:

1. Input Validation:

- Verify that both the encryption key and plain text data are present.
- If either is missing, terminate the encryption process.

2. Data Serialization:

Convert the input data into a string format.

- If the data is already a string, use it directly.
- Otherwise, serialize the data using JSON encoding.

3. AES-GCM Initialization:

Initialize the AES-GCM cipher using the provided Vault Key.

A secure random Initialization Vector (IV) is automatically generated by the cryptographic library.

4. Encryption Process:

Encrypt the serialized plaintext using AES-256-GCM.

During encryption:

- Ciphertext is produced.
- An authentication tag is generated for integrity verification.

5. Return Encrypted Output

Return the encrypted vault data along with the IV and authentication tag for secure storage or transmission.

Decryption Steps:

Input:

- encryptedVault – Ciphertext
- vaultKey – 256-bit Vault Key derived on the client
- iv – Initialization Vector used during encryption
- tag – Authentication tag

Output:

- decryptedVault – Original plaintext data

Steps:

1. AES-GCM Initialization

- Initialize the AES-GCM cipher using the Vault Key and the provided IV.

2. Decryption and Authentication

- Decrypt the ciphertext using AES-GCM.
- During this process, the authentication tag is automatically verified.

3. Integrity Verification

If the authentication tag does not match, decryption fails and no plaintext is returned, preventing tampered data from being accessed.

4. Return Plaintext

If authentication succeeds, return the decrypted plaintext vault data.

3.6.2 Argon2id Hashing Algorithm:

- Import Required Modules – Use a cryptography library that supports Argon2id (e.g., argon2-cffi in Python, or golang.org/x/crypto/argon2 in Go).
- Generate Salt – Create a random, unique salt (e.g., 16 bytes) for each password hashing operation to prevent rainbow table attacks.

- Set Argon2id Parameters – Define parameters for memory cost (e.g., 64 MB), time cost (iterations, e.g., 3), parallelism (threads, e.g., 4), and desired output hash length (e.g., 32 bytes).
- Hash Password with Argon2id – Run the Argon2id function with the password, salt, and parameters to produce a secure hash.
- Format Output Hash String – Encode the output as a string that includes all parameters, salt, and the hash (often in the format: \$argon2id\$v=19\$m=65536,t=3,p=4\$<salt_base64>\$<hash_base64>).
- Return Hashed Password String – This string is stored securely in a database.

3.6.3 XGBoost Classifier

XGBoost (eXtreme Gradient Boosting) is a highly efficient and scalable machine learning algorithm designed for supervised learning tasks, particularly classification and regression on structured/tabular data. The algorithm builds an ensemble of decision trees sequentially, where each new tree attempts to correct errors made by previous trees using gradient boosting and second-order optimization (utilizing both gradients and Hessians). XGBoost incorporates regularization (L1 and L2), handles missing values natively, and offers multiple split-finding methods, making it robust, fast, and accurate for a wide range of predictive modeling tasks.

- XGBoost minimizes a regularized objective function:

$$\text{Obj} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

Where L is the loss (e.g., logistic loss for classification), and Ω is a regularization term penalizing model complexity. The loss is approximated using Taylor expansion.

The steps in the algorithm include:

- Start with an initial prediction for all samples (e.g., log odds for binary classification).
- Calculate the first (gradient) and second (Hessian) derivatives of the loss function for each training sample.

- Fit a new decision tree to the negative gradients, using gradients and Hessians as weights.
- For each leaf node, compute the optimal output value using the objective function.
- Add the new tree's predictions to the ensemble, scaled by the learning rate (η).
- After all boosting rounds, sum the outputs of all trees to get the final prediction.

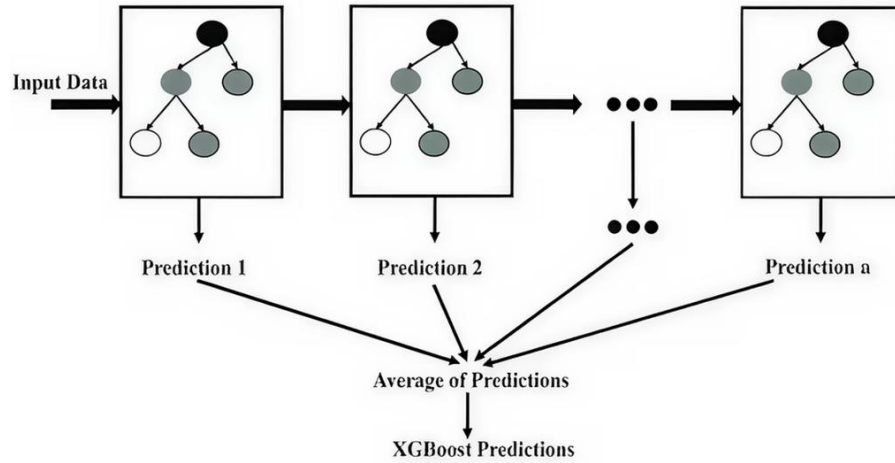


Figure 3.6.3 : XGBoost Classifier

3.6.4 SHAP Algorithm

Goal: Compute Shapley values (ϕ_i) for each feature in a tree-based model to explain individual predictions.

Algorithm Steps:

1. Input Model & Instance:
 - Provide a trained tree-based model f (e.g., XGBoost).
 - Select the instance x for which feature contributions will be calculated.
2. Define Shapley Value Formula:
 - Use the Shapley value definition:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (M - |S| - 1)!}{M!} [f(S \cup i) - f(S)]$$

Where M is the number of features and S is a subset excluding i .

3. Traverse Decision Trees:
 - For each tree in the model:
 - Traverse all paths from root to leaves.

- Track path probabilities for each subset of features encountered.

4. Compute Path Probabilities:

- For each node j in the path:

$$p_j = \prod_{k \in \text{path to } j} p_k$$

Where p_k is the split fraction at node k .

5. Calculate Feature Contributions:

- For each feature i used in a split:

$$\Delta \phi_{ij} = p_j \cdot (v_{\text{left}(j)} - v_{\text{right}(j)})$$

Where $v_{\text{left}(j)}$ and $v_{\text{right}(j)}$ are the expected values of the child nodes.

6. Aggregate Contributions:

- Sum contributions across all nodes and trees:

$$\phi_i = \sum_{t=1}^T \sum_{j \in \text{nodes}(t)} \Delta \phi_{i,j}$$

Where T is the number of trees.

7. Output Final Decomposition:

- Express the model prediction as:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

Where ϕ_0 is the baseline model prediction (average output without features).

To improve interpretability, Tree SHAP (SHapley Additive exPlanations) is integrated with XGBoost, enabling feature-level explanations for each prediction. This allows the system to show users which factors such as length, character diversity, or sequence patterns most influenced their password's strength score, thereby increasing transparency and trust in automated evaluations.

3.7 TOOLS USED

3.7.1 React Native

React Native is an open-source framework that helps to build a single JavaScript codebase, and it renders truly native UI components that run on both Android and iOS. It provides platform-agnostic primitives that map directly to native controls, enabling “learn once, write anywhere”[21]. It enables the development of native Android interfaces for the KeyShard mobile application.

3.7.2 Go

Go is a general-purpose, statically typed, compiled language from Google designed for simplicity, safety, and high concurrency. It is strongly typed and garbage-collected and has explicit support for concurrent programming. Programs are constructed from packages, whose properties allow efficient management of dependencies[22]. It is used to implement the distributed backend services and secure communication between system nodes.

3.7.3 Python

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python’s elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms; ideal for rapid prototyping and ML development[23]. In this project, Python is used for developing and training machine learning models for password strength analysis.

3.7.4 Docker

Docker is an open platform for developing, shipping, and running applications in containers (lightweight, isolated environments) that bundle code and dependencies. Docker Compose builds on this by allowing multi-container application definitions in YAML, so you can spin up linked services (databases, APIs, UIs) with a single command[24]. It is used to containerize backend services and distributed nodes, ensuring scalability and consistency.

3.7.5 Jupyter Notebook

Jupyter Notebook is an open-source interactive computing environment widely used for data analysis and machine learning development. It allows developers to write and execute code alongside visualizations and documentation[25]. In KeyShard, Jupyter Notebook is used for exploratory data analysis, feature engineering, and training and evaluating machine learning models for password strength prediction.

3.7.6 Android Studio

Android Studio is the official Integrated Development Environment (IDE) for Android application development. It provides tools for building, testing, debugging, and profiling Android applications[26]. In this project, Android Studio is used for testing and debugging the Android version of the KeyShard mobile application, ensuring secure performance, compatibility, and proper integration of platform-specific features such as authentication and secure storage.

3.7.7 VS Code (IDE)

Visual Studio Code is a free, cross-platform code editor developed by Microsoft. It supports a wide range of programming languages including JavaScript, Python, and Go, and features integrated Git, intelligent code completion, debugging tools, and a powerful extension marketplace. VS Code is widely used in full-stack development and works seamlessly with Go, Python, React Native and Docker, making it a perfect fit for this project[27].

3.7.8 Redis

Redis is an open-source, in-memory data structure store widely used as a cache, message broker, and session store. It supports a rich set of data structures including strings, hashes, lists, sets, and sorted sets, and is known for its extremely low-latency read and write operations. In KeyShard, Redis is used to implement the IP-based and email-based rate limiting mechanism, tracking request counts within rolling time windows to throttle brute-force login attempts, credential stuffing, and rapid API abuse, returning HTTP 429 responses once thresholds are exceeded.

3.7.9 Ngrok

Ngrok is a cross-platform reverse proxy and tunneling tool that creates secure public URLs for locally running services, enabling external access without requiring

deployment to a remote server. It supports HTTP, HTTPS, and TCP tunnels and provides a real-time inspection dashboard for monitoring traffic. In KeyShard, Ngrok is used during development and testing to expose the Go backend API server running on a local machine to the React Native mobile client, allowing the Android application to communicate with the backend over a public HTTPS endpoint without requiring a production deployment.

3.7.10 Deployment Environment

The server-side deployment environment for KeyShard is Linux-based. Each distributed node runs inside a Docker container built on a Debian Linux base image (debian:bookworm-slim), which is a lightweight Debian distribution suitable for production container workloads. The Go binary is compiled inside a golang:1.24 build stage and transferred into this runtime image, keeping the final container minimal and secure.

3.8 SYSTEM DEVELOPMENT & IMPLEMENTATION

3.8.1 Secure User Authentication and Vault Management

The system implements a zero-knowledge security model, ensuring that sensitive user secrets are never stored or exposed in plaintext.

3.8.1.1 User Registration (Sign-Up)

- When a user creates an account, an empty vault is initialized on the backend.
- The user's master password is processed locally on the client using Argon2id, a memory-hard key derivation function.
- The derived 256-bit Master Key is split using HKDF into:
 - VaultKey: used only for encrypting and decrypting the vault.
 - UserKey: used for authentication.
 - PrivateKey: used for authorization.
- The UserKey is hashed using SHA-256 to produce UserHash, which is stored in the database.
- Ed25519 uses the private key to generate a public key, which is stored in database.
- The vault is immediately encrypted using AES-256-GCM, ensuring confidentiality and integrity from the moment of creation.
- The raw password, master key and privatekey are never stored.

3.8.1.2 User Login and Authentication

The login process employs a robust, two-layer authentication mechanism designed to prevent unauthorized access and ensure the vault remains confidential and unviewable by any party, including the service provider.

First Layer: Email Verification This layer confirms the existence of a corresponding user account and encrypted vault.

- The user's provided email address is validated against the system's user directory.
- If no active account is associated with the email, the process is terminated immediately, and no further communication or information regarding the vault status is transmitted.

Second Layer: UserHash Authentication and Vault Retrieval This is the cryptographic core of the login, verifying the user's identity based on their master password without ever exposing the password itself.

1. **Salt Transmission:** Upon successful email verification, the backend securely transmits a unique, cryptographically random salt to the client-side. This salt prevents pre-computation attacks like rainbow tables and ensures the Master Key derivation is unique for every user.
2. **Client-Side Master Key Generation:** The received salt is combined with the user's master password and passed through Argon2id to securely generate a 256-bit Master Key. This entire operation occurs locally on the client machine.
3. **Key Derivation via HKDF:** The Master Key is split into following keys:
 - **VaultKey (256-bit):** Used for decrypting the vault.
 - **UserKey (256-bit):** Hashed using SHA-256 to create the UserHash.
 - **PrivateKey (256-bit):** Used for authorization through digital signature.
4. **Identity Authentication:** The client-derived UserHash is transmitted to the backend and compared against the stored UserHash in the database.
5. **Vault Delivery:** Only if the UserHash matches, the backend delivers:
 - The encryptedVault ciphertext.
 - The Initialization Vector (IV).
 - The Authentication Tag (Tag).
6. **Vault Decryption:** The client uses the VaultKey together with the IV and Tag to decrypt the vault using AES-GCM. All three components (VaultKey, IV, and Tag) must perfectly match the values used during encryption for the vault to be successfully and securely decrypted.

This design ensures that only the legitimate user can decrypt the vault, even the server cannot access user data.

3.8.1.3 Authorization Layer

Once the user has accessed the vault, the user needs to be authorized with the server to perform any action.

- During login, the user's private key is stored in the client's session.
- If the client requests any operation to the server, the server sends a challenge to the client. [Challenge = Random Number generated using the Operating System]
- The challenge is signed using the user's private key and sent to the server.
- The server verifies the signature using the user's public key.
- If the signature is verified, then only the server allows the action to be performed.

3.8.1.4 Secure Session Management

Session management is designed with a "zero-trust" principle to minimize the risk of residual sensitive data exposure.

- **Secure Session Termination:** When the application state changes such as a user actively logging out, a session expiring due to inactivity, or the device being powered off, the active session is instantly and securely terminated.
- **Sensitive Data Wiping:** Critical cryptographic material, including the VaultKey, Tag, and IV, along with the Master Key and any other decrypted data temporarily held in volatile memory (RAM), are securely and immediately wiped. This ensures that the data is not recoverable via memory forensics, requiring the user to complete the full login process to regain access. This prevents memory-based and cold-boot attacks.

3.8.2 Distributed Database Implementation (Go Backend)

A distributed key-value database was implemented using Go, focusing on sharding, request routing, and modular architecture.

3.8.2.1 Initializing the Golang project

To enable dependency management and version control, Go modules were initialized using:

```
PS C:\Users\sachi\Keyshard> go mod init Keyshard
>>
go: creating new go.mod: module Keyshard
```

This creates two important files:

i. **go.mod:**

Contains:

- The module path (used for imports).
- The Go version used for development.
- Required third-party libraries with specific versions.

```
1  module Keyshard
2
3  go 1.24.3
4
5  require (
6      github.com/buraksezer/consistent v0.10.0
7      github.com/cespare/xxhash/v2 v2.3.0
8      go.etcd.io/bbolt v1.4.2
9      gopkg.in/yaml.v2 v2.4.0
10 )
11
12 require golang.org/x/sys v0.29.0 // indirect
```

- ii. **go.sum:** The go.sum file stores cryptographic checksums of all module dependencies and their transitive dependencies. It is used to verify the integrity and authenticity of downloaded modules.

Each entry in go.sum contains:

- Module path.
- Module version.
- Hash of the module content.

```
go.sum
1  github.com/buraksezer/consistent v0.10.0 h1:hqBgZ1PvNLC5rkWcEBVAL9dFMBWz6I0VgUCW25rrZlU=
2  github.com/buraksezer/consistent v0.10.0/go.mod h1:6BrVajWq7wbKZlTOUPs/XVfR8c0maujuPowduSpZqmw=
3  github.com/cespare/xxhash/v2 v2.3.0 h1:UL815xU9SqsFlibzuggzjXhog7bL6oX9BbNZnL2UFvs=
```

When new packages are imported in the source code, Go automatically updates the go.mod file with the required dependency and its version. This process is managed using the command:

```
PS C:\Users\sachi\Keyshard> go mod tidy
```

3.8.2.2 Creating the standard project structure

After initializing the Golang module, the next important step was to design and create a standard project structure. A well-organized project structure is essential for

maintainability, scalability, and clarity, especially when developing a distributed system.

cmd/keyshard-node/main.go: Entry point of the database node that initializes configuration, storage, sharding, replication, and starts the HTTP server.

data/: Stores persistent data files for each database node.

deploy/node.Dockerfile: Defines the Docker image used to build and run a Keyshard node.

internal/heartbeat/: Implements heartbeat mechanisms to monitor node health and detect failures.

internal/replication/: Handles data replication logic to ensure fault tolerance and availability.

internal/utils/: Contains utility functions such as configuration loading and validation.

pkg/api/: Implements HTTP API endpoints for client interactions (GET, SET, UPDATE, DELETE).

pkg/cluster/: Manages cluster membership and node coordination.

pkg/shard/: Implements consistent hashing for distributing keys across nodes.

pkg/store/: Provide the storage abstraction and BoltDB-based persistent key-value storage.

pkg/transport/: Handles inter-node communication and request forwarding.

docker-compose.yml: Orchestrates multiple database nodes for local or containerized deployment.

go.mod: Defines the Go module name and manages project dependencies.

go.sum Stores checksums to ensure dependency integrity and reproducible builds.

node1.yaml – node5.yaml: Configuration files defining individual node settings and peer information.

README.md: Documents the project overview, set up instructions, and usage details.

3.8.2.3 Creating the entry point(main.go)

The main.go file serves as the primary entry point of the Keyshard distributed database node. It is responsible for initializing the application by parsing command-line arguments, loading node configuration from a YAML file, and applying environment variable overrides for containerized deployment. The file sets up the consistent hashing ring using peer node information, initializes the local persistent storage using BoltDB, and creates the API server instance. Finally, it registers HTTP routes for key-value operations and starts the HTTP server, enabling the node to participate in the distributed database cluster.

This main.go file reads config path from command line.

```
// Read config file path
configPath := flag.String("config", "", "Path to node config YAML")
flag.Parse()

if *configPath == "" {
    log.Fatal("config file path is required")
}
```

Loads YAML configuration

```
// Load YAML config
cfg, err := utils.LoadConfig(*configPath)
if err != nil {
    log.Fatal("failed to load config:", err)
}
```

Overrides config with Docker env variables if present

```
// Override with Docker environment variables if present
if envNodeID := os.Getenv("NODE_ID"); envNodeID != "" {
    cfg.NodeID = envNodeID
}
if envNodeAddr := os.Getenv("NODE_ADDR"); envNodeAddr != "" {
    cfg.HTTPAddr = envNodeAddr
}
```

Builds shard members from peer list


```
// Build shard members from peers
var members []shard.Member
for _, peer := range cfg.Peers {
    members = append(members, shard.Member{
        Name:    peer.ID,
        Address: peer.Address,
    })
}
```

Creates consistent hash ring

```
// Create consistent hash ring
ring := shard.NewRing(members)
```

Opens local BoltDB database

```
// Open local BoltDB
db, closeFn, err := store.NewDatabase(cfg.DBLocation)
if err != nil {
    log.Fatal(err)
}
defer closeFn()
```

Creates API server:

```
server := api.NewServer(db, ring, cfg.NodeID)
```

Registers HTTP routes

```
// Routes
http.HandleFunc("/get", server.GetHandler)
http.HandleFunc("/set", server.SetHandler)
http.HandleFunc("/delete", server.DeleteHandler)
http.HandleFunc("/update", server.UpdateHandler)
```

Starts HTTP server and listen

```
log.Printf("Node %s listening on %s", cfg.NodeID, cfg.HTTPAddr)
log.Fatal(http.ListenAndServe(cfg.HTTPAddr, nil))
```

3.8.2.4 Configuration driven design setup

A configuration-driven design allows your system to change behavior, parameters, or runtime settings without modifying the code, by reading external configuration files or environment variables. Our project consists of:

YAML Config Files:

Each node has a .yaml file (node1.yaml, node2.yaml, etc.) defining:

- node_id: unique identifier for the node.
- http_addr: address and port of http address.
- db_location: path to the nodes local database.
- peers: list of other nodes in cluster with their ids and addresses.

```
! node1.yaml > {} peers > {} 4
1  node_id: node1
2  http_addr: "0.0.0.0:8006"
3  db_location: "/app/data/node1.db"
4
5  peers:
6    - id: node1
7      address: "node1:8006"
8    - id: node2
9      address: "node2:8007"
10   - id: node3
11     address: "node3:8008"
12   - id: node4
13     address: "node4:8009"
14   - id: node5
15     address: "node5:8010"
```

Environment variables override (for docker):

This allows dynamic deployment inside docker containers without editing YAML. It contains only node_id and node_addr as shown in main.go override config.

Sharding and peer awareness:

Node peers are read from config, used to build shard members which allows consistent hashing ring creation without hardcoding addresses. The major benefits of this are:

- No code change for deployment or scaling.
- Easy to update ports,db paths or cluster topology.
- Supports dynamic environment deployment(local, docker, cloud).

3.8.2.5 Implementing the storage module

The storage module is a part of the system that stores data, i.e. key-value pairs.

1. Entry struct:

Represents a single key-value record in your database. It contains:

- Key: the identifier for the record.
- Value: the data itself.
- Metadata: optional extra info.

```
// Entry is one vault record
type Entry struct {
    Key      string          // e.g. "user123:entry456"
    Value    []byte           // encrypted payload
    Metadata map[string]string // optional JSON metadata
}
```

2. Store interface:

Defines what a storage module must do (CRUD operations). Any backend storage that implements this interface can be used interchangeably.

```
// Store represents the KV interface
type Store interface {
    Put(e *Entry) error
    Get(key string) (*Entry, error)
    Delete(key string) error
}
```

3.8.2.6 Implementing sharding logic

Sharding logic is the mechanism that decides which node should store a given key in a distributed system like KeyShard. In our project sharding means the key space is divided and each key is assigned one special node. Node collectively form a cluster.

Shard member definition

Here, each shard node is represented as a member containing:

- Name: Unique identifier of the node.
- Address: Network address used for internode communication.

```
// Member defines a shard node that participates in the ring
type Member struct {
    Name    string
    Address string
}
```

Consistent hash ring structure

The Ring structure wraps the consistent hashing engine, abstracting the hashing logic from the rest of the system.

```
// Ring wraps the consistent hashing implementation
type Ring struct {
    c *consistent.Consistent
}
```

This design ensures clean separation of concerns and enables easy extension.

Ring initialization

Ring initialization is done using the newRing function in our project:

```
// NewRing creates a new consistent hash ring with given members
func NewRing(nodes []Member) *Ring {
    cfg := consistent.Config{
        PartitionCount: 271, // number of virtual nodes
        ReplicationFactor: 20, // redundancy factor
        Load: 1.25, // balancing factor
        Hasher: hasher{},
    }

    members := make([]consistent.Member, len(nodes))
    for i, n := range nodes {
        members[i] = n
    }

    c := consistent.New(members, cfg)
    return &Ring{c: c}
}
```

Here, the NewRing function initializes a consistent hash ring using the list of available shard nodes. It prepares the system to deterministically map keys to nodes for distributed storage.

- It configures the hash ring with virtual nodes, replication factor, and load balancing.
- It wraps each physical node into a member to participate in the ring.
- Creates a consistent hash ring using the consistent library.
- Maps keys to nodes deterministically for balanced sharding.
- Returns a Ring object that provides key lookup and node management functions.

Hash function implementation

Hash function is the core of sharding logic.

```
// hasher defines a fast 64-bit xxhash hasher
type hasher struct{}

func (h hasher) Sum64(data []byte) uint64 {
    return xxhash.Sum64(data)
}
```

- Uses xxHash which is extremely fast and low collision.
- Converts keys into numeric hash values.
- Ensures deterministic and uniform distribution.

Key to node mapping:

It ensures same key always maps to same node. Minimal disruption when node changes.

```
// Find which node owns a key
func (r *Ring) Lookup(key string) (*Member, error) {
    m := r.c.LocateKey([]byte(key))
    mem, ok := m.(Member)
    if !ok {
        return nil, fmt.Errorf("invalid member type")
    }
    return &mem, nil
}
```

Its working:

- The client provides a key.
- The key is hashed using xxHash.
- The hash is mapped to a position on the ring.
- The nearest shard node clockwise owns the key.

3.8.2.7 Implementing HTTP API layer

Our project implements HTTP API layer in the following way:

- i. Server struct wraps the database (Store interface), consistent hash ring, and node ID.

```
type Server struct {
    db      store.Store
    ring    *shard.Ring
    nodeID  string
}
```

- ii. NewServer() initializes the server with the storage and sharding info.

```
func NewServer(db store.Store, ring *shard.Ring, nodeID string) *Server {  
    return &Server{  
        db:    db,  
        ring:  ring,  
        nodeID: nodeID,  
    }  
}
```

- iii. routeOrHandle() decides whether a key operation should be handled locally or forwarded to the node owning the key.

```
27  
28 func (s *Server) routeOrHandle(  
29     w http.ResponseWriter,  
30     r *http.Request,  
31     key string,  
32     localHandler func() error,  
33 ) {  
34 >     if key == "" { ...  
37     }  
38  
39     owner, err := s.ring.Lookup(key)  
40 >     if err != nil { ...  
43     }  
44  
45     if owner.Name == s.nodeID {  
46         err := localHandler()  
47         fmt.Fprintf(w, "Node=%s Error=%v", s.nodeID, err)  
48         return  
49     }  
50  
51     target := fmt.Sprintf("http://%s%s%s",  
52         owner.Address,  
53         r.URL.Path,  
54         r.URL.RawQuery,  
55     )  
56  
57     resp, err := http.Get(target)  
58 >     if err != nil { ...  
61     }  
62     defer resp.Body.Close()  
63  
64     body, _ := io.ReadAll(resp.Body)  
65     w.WriteHeader(resp.StatusCode)  
66     w.Write(body)  
67 }  
68
```

- iv. Handlers (GetHandler, SetHandler, UpdateHandler, DeleteHandler) extract query params and perform operations via routeOrHandle.

```

> func (s *Server) GetHandler(w http.ResponseWriter, r *http.Request) { ...
}

> func (s *Server) SetHandler(w http.ResponseWriter, r *http.Request) { ...
}

> func (s *Server) UpdateHandler(w http.ResponseWriter, r *http.Request) { ...
}

> func (s *Server) DeleteHandler(w http.ResponseWriter, r *http.Request) { ...
}

```

- v. HTTP responses return operation results or errors to the client, supporting distributed request routing.

This layer acts as the front-facing REST interface for distributed KV store, handling requests, sharding logic, and forwarding.

3.8.2.8 Replication

```

// Replicate to other nodes
nodes := s.ring.AllMembers()
for _, node := range nodes {
    // Skip self
    if node.Name == s.nodeID {
        continue
    }

    go func(nodeAddr string) {
        client := &http.Client{Timeout: 2 * time.Second}

        // Replicate userHash key
        req1, _ := http.NewRequest(
            http.MethodPost,
            fmt.Sprintf("http://%s/replicate", nodeAddr),
            bytes.NewReader(data),
        )
        req1.Header.Set("X-Key", "user:"+user.UserHash)
        req1.Header.Set("Content-Type", "application/json")
        client.Do(req1)

        // Replicate email key
        req2, _ := http.NewRequest(
            http.MethodPost,
            fmt.Sprintf("http://%s/replicate", nodeAddr),
            bytes.NewReader(data),
        )
        req2.Header.Set("X-Key", "email:"+user.Email)
        req2.Header.Set("Content-Type", "application/json")
        client.Do(req2)
    }(node.Address)
}

```

The flow of replication is:

1. Fetch all the nodes.
2. Skip the current node where data has already been stored locally.
3. Asynchronous replication occurs through a thread like go routine that helps replication in background so that user doesn't have to wait.
4. Sends http request to store both primary index (userhash) and secondary index (email) using replicate handler.
5. Each node gets the request and stores data in their local database

3.8.2.9 Dockerization of the project

The main purpose of using docker is isolation, consistency, easy scaling and portability. Our project contains dockerfile, configuration via docker and docker compose for multi node setup.

Dockerfile

It is used to create a container image for database nodes.

It has two stages:

i. Building the go program:

```
FROM golang:1.24 AS builder

WORKDIR /app

COPY go.mod go.sum ./
RUN go mod download

COPY ./cmd/keyshard-node ./cmd/keyshard-node
COPY ./pkg ./pkg
COPY ./internal ./internal

RUN go build -o keyshard ./cmd/keyshard-node
```

- a. FROM golang:1.24 AS builder: starts from a go programming language environment.
- b. WORKDIR /app: sets /app as the working folder inside the container
- c. Copies go dependencies and downloads all necessary go libraries inside the container making sure that program can be built with all dependencies.
- d. Copies the actual program code into the container and compiles the go code into a single executable file called keyshard.

ii. Runtime image

```
FROM debian:bookworm-slim

WORKDIR /app

COPY --from=builder /app/keyshard /app/keyshard

# Copy node configs
COPY ./node1.yaml ./node1.yaml
COPY ./node2.yaml ./node2.yaml
COPY ./node3.yaml ./node3.yaml
COPY ./node4.yaml ./node4.yaml
COPY ./node5.yaml ./node5.yaml

# Create data folder
RUN mkdir -p /app/data

# Expose all possible internal ports
EXPOSE 8006 8007 8008 8009 8010

# Entry point
ENTRYPOINT ["/keyshard"]
```

1. Starts a very small linux operating system to run the program.
2. Set the working folder inside the container.
3. Copies the KeyShard program we built in stage 1 into smaller runtime images.
4. Copies configuration files of all nodes present into the container.
5. Creates a folder to store the database files inside the container where each node will store its BoltDB database.
6. Expose tells docker which ports may be used by the program\.
7. Entrypoint sets the default program to run when the container starts that means when we start the container, it automatically runs the Keyshard node.

Docker compose

This docker-compose.yml file is used to run all 5 KeyShard nodes together as separate containers. Docker Compose helps manage multiple containers at once, instead of starting each node manually. Here , each node is a separate service/container running one KeyShard node.

```

docker-compose.yml - The Compose specification establishes a
1  services:
    ▷Run Service
2  node1:
3    build:
4      context: .
5      dockerfile: deploy/node.Dockerfile
6    container_name: keyshard_node1
7    ports:
8      - "8001:8006"
9    environment:
10     NODE_ID: "node1"
11     NODE_ADDR: "0.0.0.0:8006"
12    volumes:
13     - ./data/node1:/app/data
14    command: ["-config=node1.yaml"]
15
16  > node2: ...
29
30  > node3: ...
43
44  > node4: ...
57
58  > node5: ...
71

```

Let us consider node1 as an example and understand the process step by step:

- a. services: Defines all containers that make up the KeyShard application.
- b. node1: Represents individual database nodes in the distributed system.
- c. build: Instructs Docker Compose to build a custom image for the service.
- d. context: Specifies the project root directory used during image build.
- e. dockerfile: Points to the Dockerfile used to build the KeyShard image.
- f. container name: Assigns a fixed, readable name to the running container.
- g. ports: Maps a host machine port to a container port for external access.
- h. environment: Sets environment variables inside the container.
- i. volumes: Mounts host directories into containers for persistent storage.
- j. command: Passes runtime arguments to the application binary.

Health check and automatic node restart:

```
node5:
  build:
    context: .
    dockerfile: deploy/node.Dockerfile
  container_name: keyshard_node5
  restart: unless-stopped
  ports:
    - "8005:8010"
  environment:
    NODE_ID: "node5"
    NODE_ADDR: "0.0.0.0:8010"
  volumes:
    - ./data/node5:/app/data
  command: ["-config=node5.yaml"]
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8010/health"]
    interval: 10s
    timeout: 3s
    retries: 3
    start_period: 5s
```

Here, the node is run inside the docker container. Hence, we use docker features to handle node failure and health checking.

Here, `restart: unless-stopped` means until the node is not stopped manually, the docker restarts in case of any node failure. Similarly, we use API call to test whether the node is running or not in regular interval. Each node sends heartbeat ping to its peers which helps filter failed nodes.

3.8.3 Machine Learning–Based Password Strength Analysis

3.8.3.1 Dataset Preparation

For Dataset, the RockYou dataset was used, which is a real world password leak dataset. The passwords are stored in plain text, and the file consists of about 14 million passwords.

Since the dataset consists of real passwords, data cleaning was done to remove non-printable characters, strip white space and drop nulls and duplicates. The non-printable characters were targeted from ASCII ranges `\x00-\x1F` and `\x7F-\x9F`.

A function to get the password strength was defined by using the combination of two libraries `zxcvbn` and `PasswordStats`. While `zxcvbn` generates strength using factors like length, dictionary, patterns etc, `PasswordStats` is more entropy based, so using both balances out the strength. The function was used to generate password strength for all the passwords in the dataset, and a balanced dataset of 600,000 passwords, containing 200,000 passwords for each class (weak, medium and strong) was created.

3.8.3.2 Feature Engineering

The next step was to extract features from the password dataset. A total of 14 features were extracted which are as follows:

- i. **Length:** Total number of characters in the password.
- ii. **Lowercase_count:** Number of lowercase alphabetic characters(a-z).
- iii. **Uppercase_count:** Number of uppercase alphabetic characters(A-Z).
- iv. **Digit_count:** Number of numeric characters(0-9).
- v. **Special_count:** Number of non-alphanumeric characters such as punctuation and symbols.
- vi. **Entropy:** The Shannon entropy computed from character frequency distribution, measuring unpredictability of the character composition. For a password of length n , each unique character's probability p is calculated as its frequency divided by n .

$$H = -\sum p \cdot \log_2(p)$$

For e.g. “aaaaa” would score 0, and “aB3!xa%” would score more, due to high character variety.

- vii. **Transitions:** It defines the number of adjacent character type changes capturing structural mixing throughout the password. Each character in the password is mapped to one of four types: lowercase(l), uppercase(u), digit(d) or special(s). E.g. aaB3 has 2 transitions.
- viii. **Max_repeat:** It defines the length of the longest consecutive run of identical characters, penalizing repetitive sequences. It uses a regular expression to find all runs of consecutive identical characters, then returns the length of the longest run. For e.g. “aaaabbc” has max repeat 4.
- ix. **Leet_count:** Count of common leet substitution characters such as @,&!,0,1,3 etc, measuring obfuscation attempts. For e.g. writing pattern as p@ttern
- x. **Is_common_root:** It is a binary flag indicating presence of a known weak root word such as password, abcd, qwerty within the password.

- xi. Shape_complexity:** It defines the number of unique two character type bigrams with U (uppercase), L (lowercase), D (digit) and S (special). All unique consecutive two character bigrams are then extracted and counted. For e.g. aabbccDD has the shape of “LLLLLUU”. The extracted bigram would be “LL”, “LU”, and “UU”, and thus the shape complexity would be 3, while transition would be just 1.
- xii. Pattern_intensity:** It counts the forward and backward keyboard walk and alphabetic sequence fragments of length three or more. The sequences are the full alphabet, digits, the three keyboard rows like qwertyiop. For e.g. qwerty123 has a score of 5 (qwe, wer, ert, rty and 123)
- xiii. Habit_score:** It is a composite score from 0 to 3, reflecting common user habits such as starting with uppercase, ending with a digit and containing years. For e.g. Password2024 has a score of 3
- xiv. Char_diversity:** It is the total count for lowercase, uppercase, digit and special digits in binary form. For e.g. abc2 has char_diversity of 2.

3.8.3.3 Model Training and Evaluation

XGBoost classifier was used for strength prediction. The features were divided into two parts, with one consisting of previously listed features, and the other with the feature strength, i.e. the parameter to be predicted.

The dataset was split across three ways: train, validation and test. First the dataset is divided into 80% training set, and remaining 20% is for test dataset using stratify. Similarly, the next split is done in the remaining 80% training set, where 15% is split into validation set, and the remaining 85% is for the training set. Thus, the total split looked like:

Train:	407,522
Validation:	71,916
Test:	119,860

First, a baseline prediction with minimum parameters was done, achieving a macro average of 87.06%. then, hyperparameter tuning was conducted with RandomizedSearchCV, fitting 3 folds with 15 random combinations. Finally, the computed parameters were used to train the final model, with following report:

Class	Precision	Recall	F1-Score	Support
Weak	0.9062	0.7006	0.7902	39991
Medium	0.7539	0.9185	0.8281	39974
Strong	0.99	0.9985	0.9942	39895
Macro Average	0.8834	0.8725	0.8709	119860
Weighted Average	0.8833	0.8724	0.8708	119860

We can see that the recall for class weak is less, thus to increase it, we use a custom probability threshold. This is possible because while training the model, we used multi:softprob, which outputs probability from which class is selected rather than selecting the class directly.

To balance the medium class as well, the best threshold was found to be 0.41. The report generated was as follows:

Class	Precision	Recall	F1-Score	Support
Weak	0.8050	0.8020	0.8035	39991
Medium	0.8006	0.7967	0.7987	39974
Strong	0.9900	0.9985	0.9942	39895
Macro Average	0.8652	0.8657	0.8655	119860

Finally, using the model, the shap library was used to extract shapley values.

3.8.3.4 Integration

The final model was exported as a pkl, onnx and json file. The SHAP recommendation json file was updated with required priority update and message. Using a library called onnx-runtime, specifically built for react native, the model was loaded in the frontend. feature extraction logic was copied exactly as done in python, and inference was connected to required areas.

3.9 VERIFICATION AND TESTING

The verification and testing process for KeyShard was structured around the following principles:

- Every cryptographic primitive must produce deterministic, correct, and tamper evident outputs.

- The distributed storage layer must correctly route, store, forward, and replicate data across nodes.
- The ML classifier must generalize to unseen passwords with measurable accuracy.
- The mobile application must enforce zero-knowledge guarantees at every user interaction point.
- All system components must integrate correctly in the Dockerized deployment environment.

3.9.1 Verification Strategy

Verification was conducted at four levels; each employing methods appropriate to the component under test. The complete verification strategy is summarized below:

Table 3.9.1: Verification Strategy Matrix

Level	Component	Verification Method	Tools Used
Unit	Argon2id KDF	Deterministic output, byte-length, salt uniqueness	Go test framework
Unit	HKDF Key Separation	Key distinctness, byte-level comparison	Go test framework
Unit	AES-GCM Encryption	Round-trip correctness, tamper detection	Go test framework
Unit	SHA-256 UserHash	Fixed-length output, one-way behavior	Go test framework
Unit	xxHash Sharding	Deterministic key-to-node mapping	Go test framework
Component	REST API Endpoints	Request/response validation, status codes	cURL, Postman
Component	BoltDB Storage	Persistence across restarts, CRUD correctness	cURL, Docker
Component	ML Feature Extraction	Feature count, value range, no data leakage	Python unittest
Component	XGBoost Classifier	Accuracy, F1-score, ROC-AUC on test set	Scikit-learn

Component	SHAP Explainer	Feature attribution sum equals prediction delta	SHAP library
Integration	Auth + Vault Flow	End-to-end registration and login	React Native test env
Integration	Distributed CRUD	Multi-node operations, forwarding correctness	Docker Compose
Integration	ML + UI	Real-time strength feedback in credential entry	Manual testing
Security	Tamper Detection	Modified ciphertext decryption attempt	Manual testing
Security	Auth Bypass	Invalid credentials, replayed tokens	Manual testing
Security	Rate Limiting	Rapid repeated requests	cURL scripting

3.9.2 Cryptographic Verification

The cryptographic stack of KeyShard forms the security foundation of the entire system. Verification of each cryptographic component was conducted independently before integration testing.

3.9.2.1 Argon2id Master Key Derivation Verification

The Argon2id function was configured with the following parameters for all verification tests:

- Memory cost: 64 MB (65536 KB)
- Time cost: 3 iterations
- Parallelism: 4 threads
- Output length: 32 bytes (256 bits)

Test 1. Output Length Correctness: The derived master key was inspected as a Uint8Array. The array length was confirmed to be exactly 32 bytes across all test inputs, validating correct 256-bit key generation.

Test 2. Deterministic Behavior: The same email, master password, and salt were supplied to Argon2id across 10 independent invocations. All 10 outputs were identical

at the byte level, confirming that key derivation is deterministic and reproducible for identical inputs, a prerequisite for the login re-derivation flow.

Verification Output (Sample):

Verification for argon2id masterkey:

```
Unit8Array: [
    167, 250, 250, 62, 110, 178, 84, 21,
    41, 5, 39, 239, 21, 11, 87, 55,
    115, 49, 247, 186, 146, 93, 77, 163,
    31, 2, 102, 172, 139, 99, 146, 126
]

Hexadecimal: a7fafa3e6eb25415290527ef150b57377331f7ba925d4da31f0266ac8b63927e
```

Test 3. Salt Uniqueness: Fifty independent registrations were simulated, each generating a new cryptographically random 16-byte salt. SHA-256 hashing of all 50 salts confirmed zero collisions, validating that salt generation produces unique values per registration, preventing precomputation attacks.

Test 4. Cross-Salt Key Distinctness: The same master password was hashed with 50 different random salts. All 50 resulting master keys were distinct at every byte position, confirming that salt variation produces entirely different keys from the same password, the fundamental defense against rainbow table attacks.

Table 3.9.2.1: Argon2id Verification Results

Test	Condition	Expected Result	Result	Status
Output length	Any valid input	Exactly 32 bytes	32 bytes confirmed	PASS
Determinism	Same input × 10	Identical output	Byte-identical across all trials	PASS
Salt uniqueness	50 registrations	No salt collisions	0 collisions detected	PASS
Cross-salt distinctness	Same password, 50 salts	All keys distinct	All keys differ at every byte	PASS
Hexadecimal encoding	32-byte key	64-char hex string	Correct encoding confirmed	PASS

3.9.2.2 HKDF Key Separation Verification

After Argon2id derives the 256-bit master key, HKDF is applied to produce two independent derived keys: the VaultKey (for AES-GCM encryption) and the UserKey (for authentication hashing).

Test 1. Key Distinctness: For 50 distinct master keys, the VaultKey and UserKey were compared byte-by-byte. In all 50 cases, the two keys differed at every byte position, confirming that HKDF produces cryptographically independent output streams.

Verification Output (Sample):

Verification for HKDF key derivation process:

```
Unit8 Vault Key:
[45, 54, 4, 55, 73, 32, 228, 140, 176, 55, 43, 72, 48, 82, 148, 186, 224, 150, 25, 89,
214, 126, 221, 252, 12, 74, 187, 210, 64, 116, 197, 209]

Unit8 User Key:
[98, 78, 92, 73, 229, 239, 229, 255, 92, 74, 250, 87, 29, 231, 168, 244, 225, 6, 158, 84,
72, 144, 32, 100, 71, 177, 44, 171, 222, 223, 26, 229]

Sha256 User Hash:
[56, 189, 103, 146, 60, 239, 55, 93, 134, 130, 117, 65, 250, 104, 39, 235, 96, 187, 43,
213, 134, 239, 158, 253, 242, 33, 65, 225, 23, 111, 4, 177]
```

Test 2. UserKey SHA-256 Hashing: The UserKey was hashed using SHA-256, producing a fixed-length 32-byte (256-bit) output in all test cases. The one-way nature was confirmed by verifying that no two distinct UserKeys produced the same SHA-256 hash across all 50 test inputs.

Test 3. VaultKey Never Transmitted: A network traffic inspection was conducted during the login flow to confirm that the VaultKey bytes do not appear in any outbound HTTP request. Only the SHA-256 UserHash is transmitted to the backend for comparison, the VaultKey remains exclusively on the client device.

Table 3.9.2.2: HKDF Key Separation Verification Results

Test	Condition	Expected Result	Result	Status
VaultKey $\neq$ UserKey	50 distinct master keys	Keys differ at all bytes	Confirmed for all 50 trials	PASS
SHA-256 output length	Any UserKey input	Exactly 32 bytes	32 bytes in all cases	PASS
SHA-256 one-way behavior	50 distinct UserKeys	No hash collisions	0 collisions detected	PASS

VaultKey not transmitted	Network inspection	VaultKey absent in requests	Not found in any outbound request	PASS
Key reproducibility	Same master key re-derived	Same VaultKey and UserKey	Byte-identical on re-derivation	PASS

3.9.2.3 AES-GCM Encryption and Decryption Verification

AES-256-GCM was applied to encrypt the complete vault JSON object prior to storage. Verification confirmed both correct encryption behavior and the authenticated encryption property.

Test 1. Ciphertext Opacity: A structured vault JSON object containing credential fields (title, username, password, URL, recovery email) was encrypted using AES-256-GCM. Inspection of the ciphertext confirmed that no original field names, values, or structural patterns were recoverable from the encrypted output, confirming complete plaintext obscuring.

Test 2. Round-Trip Correctness: The encrypted vault was decrypted using the correct VaultKey, IV, and authentication tag. The decrypted output was compared byte-by-byte with the original plaintext across 20 independent trials. All 20 trials produced exact matches, confirming encryption and decryption correctness.

Sample Verified Output:

Verification for AES-GCM encryption and decryption:

```

Encrypted:
["0DFu0hK1BNE2anfofNcyarWVolzF+4h+uYm2y4L0z1+381t00Wtufu5s4uGgTWHf4dnYUSeuVC1o7JC502Ee537F1VU/SuBYjyrfL3fgLNoUK73oLJAKQ7Qz3o6Se112+28B1W+9E0B2T8+qC3n4P9yfs76uYf3hu4R83o26+8P+PynSu784tQ1uodpx35n6b4WE70Bfu7u63db+R18Q8B/TH/pL
fm1s1V8h0YphqzF6K8q85ZagpLudqY0uWdg7CJE2AVLFD9huw07f9fYfC13obGmS6ES5KqhgV3H83uJwU7Tgs54pKVW8HLEBPNQ7r87H+QzY+5uWYodux/r8uk3u/4L3M0LqTACm7u27710osDIErXBS+8p58SPG3uodp67u9pdu8gTheWt1Xg+nd0P9YVWu68uau0t1fjhzLQ0vrt61e211n/rzh
RLTWZKZuZu0u0zob3H38b3yZ2zHfW8R8b8L1W8b8tTNC8b8r+GACu31/c3okc6118f8U7E1K8ppl2FY2yR1FqCPTerC87PwC/rn6
+5313h+varE38u0zob3H38b3yZ2zHfW8R8b8L1W8b8tTNC8b8r+GACu31/c3okc6118f8U7E1K8ppl2FY2yR1FqCPTerC87PwC/rn6
j5c38uL6Y8b8hV7u8uLd8u1y8P0F8H8Q8H5s5Pn0K48u8L2u8T0533nF8Zj8uL8E8y8b8
+P+P3MNSu8G8u8S2u8b+onCPM8R8B1u8FV578N8b8F8L8B8Lq1898b8F8L8N8N15KQc5Y8Y8R8Y8L8V8g8X8r8Se8T8dd/g18y8h83g13Tj8h8Y8uLcvL55T8g1L8uLJC1B8r7m8pLsF8m8f8c8B/TgT8n8E8u8CA8R8Y8K8/N00H81E~"]

Decrypted:
{
  "title": "Gmail",
  "username": "ravibh2003@gmail.com",
  "password": "Testtestest",
  "url": "https://gmail.com",
  "category": "Other",
  "notes": "Hello",
  "tags": "Work,fun,imp",
  "recoveryEmail": "ravironbhussal@gmail.com",
  "recoveryPhone": "9864211111",
  "createdat": "2025-12-13T12:42:18.532Z"
}

```

Test 3. Tamper Detection: The encrypted vault ciphertext was modified by flipping a single bit at three different positions (beginning, middle, end of ciphertext). In all three cases, AES-GCM authentication tag verification failed upon decryption attempt, and no partial plaintext was returned. This confirms that any unauthorized modification of stored vault data is detected before decryption proceeds.

Test 4. Wrong Key Rejection: Decryption was attempted using a VaultKey derived from a different master password. Authentication tag verification failed in all 10 trials, confirming that vault data is inaccessible without the correct master password.

Test 5. IV Uniqueness: Fifty vault encryption operations were performed. The initialization vector (IV/nonce) generated for each operation was confirmed to be unique across all 50 trials, preventing nonce reuse, a critical requirement for AES-GCM security.

Table 3.9.2.3: AES-GCM Verification Results

Test	Condition	Expected Result	Result	Status
Ciphertext opacity	Structured JSON input	No plaintext recoverable	No patterns found in ciphertext	PASS
Round-trip correctness	20 encrypt-decrypt cycles	Exact plaintext restoration	Byte-identical in all 20 trials	PASS
Tamper detection: start	1-bit flip at ciphertext start	Decryption abort, no output	Auth tag failure, no plaintext	PASS
Tamper detection: middle	1-bit flip at ciphertext middle	Decryption abort, no output	Auth tag failure, no plaintext	PASS
Tamper detection: end	1-bit flip at ciphertext end	Decryption abort, no output	Auth tag failure, no plaintext	PASS
Wrong key rejection	Different master password	Decryption fails	Auth tag failure in all 10 trials	PASS
IV uniqueness	50 encryption operations	All IVs distinct	0 IV collisions detected	PASS

3.9.3 Distributed Storage Verification

3.9.3.1 Node Deployment Verification

Five KeyShard database nodes were deployed using Docker Compose. Each node runs as an independent container with its own BoltDB storage volume and HTTP API layer.

Docker Deployment Output (Verified):

Simulation of 5 nodes using docker yaml file:

```
PS C:\Users\sachi\keyshard> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
654c526ff2dc	keyshard-node1	"/keyshard -config=..."	4 minutes ago	Up 4 minutes	0.0.0.0:8001->8006/tcp, [::]:8001->8006/tcp	keyshard_node1
f347739320cf	keyshard-node2	"/keyshard -config=..."	4 minutes ago	Up 4 minutes	0.0.0.0:8002->8007/tcp, [::]:8002->8007/tcp	keyshard_node2
4c2943e62d4a	keyshard-node5	"/keyshard -config=..."	4 minutes ago	Up 4 minutes	0.0.0.0:8005->8010/tcp, [::]:8005->8010/tcp	keyshard_node5
2986f2e10d8a	keyshard-node4	"/keyshard -config=..."	4 minutes ago	Up 4 minutes	0.0.0.0:8004->8009/tcp, [::]:8004->8009/tcp	keyshard_node4
3e9069539fcd	keyshard-node3	"/keyshard -config=..."	4 minutes ago	Up 4 minutes	0.0.0.0:8003->8008/tcp, [::]:8003->8008/tcp	keyshard_node3

Table 3.9.3.1: Node Deployment Verification Results

Test	Condition	Expected Result	Result	Status
Port mapping	5 containers	Unique ports 8001–8010	All ports mapped correctly	PASS
Container isolation	Individual node stop	Other nodes unaffected	Remaining 4 nodes operational	PASS
Storage volume isolation	Per-container volume	No cross- container data access	Volumes isolated per container	PASS
Internal networking	Service name resolution	Nodes communicate by name	Inter-node communication confirmed	PASS
Localhost independence	Remove localhost binding	Nodes accessible via service names	No localhost dependency after containerization	PASS

3.9.3.2 API Verification Using cURL

All four core REST API operations were verified against the live 5-node cluster using cURL. The following operations were tested with results documented below.

SET Operation Verification: Confirmed successful storage of key-value data.

```
PS C:\WINDOWS\System32> curl "http://localhost:8001/set?key=user1&value=hello"
```

```

StatusCode      : 200
StatusDescription : OK
Content         : Node=node1 Error=<nil>
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 22
                  Content-Type: text/plain; charset=utf-8
                  Date: Mon, 05 Jan 2026 12:16:20 GMT
                  Node=node1 Error=<nil>
Forms           : {}
Headers         : {[Content-Length, 22], [Content-Type, text/plain; charset=utf-8], [Date, Mon, 05 Jan 2026 12:16:20 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : System.__ComObject
RawContentLength : 22

```

GET Operation Verification: Confirmed accurate retrieval of stored data.

```

PS C:\WINDOWS\System32> curl "http://localhost:8001/get?key=user1"

StatusCode      : 200
StatusDescription : OK
Content         : helloNode=node1 Error=<nil>
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 27
                  Content-Type: text/plain; charset=utf-8
                  Date: Mon, 05 Jan 2026 12:13:40 GMT

                  helloNode=node1 Error=<nil>
Forms           : {}
Headers        : {[Content-Length, 27], [Content-Type, text/plain; charset=utf-8], [Date, Mon, 05 Jan 2026 12:13:40
                  GMT]}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : System.__ComObject
RawContentLength : 27

```

UPDATE Operation Verification: Confirmed correct modification of existing entries.

```

PS C:\WINDOWS\System32> curl "http://localhost:8001/update?key=user1&value=updated"

StatusCode      : 200
StatusDescription : OK
Content         : Node=node1 Error=<nil>
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 22
                  Content-Type: text/plain; charset=utf-8
                  Date: Mon, 05 Jan 2026 12:17:02 GMT

                  Node=node1 Error=<nil>
Forms           : {}
Headers        : {[Content-Length, 22], [Content-Type, text/plain; charset=utf-8], [Date, Mon, 05 Jan 2026 12:17:02
                  GMT]}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : System.__ComObject
RawContentLength : 22

```

Post-Update GET Verification:

```

PS C:\WINDOWS\System32> curl "http://localhost:8001/get?key=user1"

StatusCode      : 200
StatusDescription : OK
Content         : updatedNode=node1 Error=<nil>
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 29
                  Content-Type: text/plain; charset=utf-8
                  Date: Mon, 05 Jan 2026 12:17:29 GMT

                  updatedNode=node1 Error=<nil>
Forms           : {}
Headers        : {[Content-Length, 29], [Content-Type, text/plain; charset=utf-8], [Date, Mon, 05 Jan 2026 12:17:29
                  GMT]}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : System.__ComObject
RawContentLength : 29

```

DELETE Operation Verification: Confirmed permanent removal of entries.

```

PS C:\WINDOWS\System32> curl "http://localhost:8001/delete?key=user1"

StatusCode      : 200
StatusDescription : OK
Content         : Node=node1 Error=<nil>
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 22
                  Content-Type: text/plain; charset=utf-8
                  Date: Mon, 05 Jan 2026 12:18:00 GMT

                  Node=node1 Error=<nil>
Forms           : {}
Headers        : {[Content-Length, 22], [Content-Type, text/plain; charset=utf-8], [Date, Mon, 05 Jan 2026 12:18:00
                  GMT]}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : System.__ComObject
RawContentLength : 22

```

Post-Delete GET Verification:

```

PS C:\WINDOWS\System32> curl "http://localhost:8001/get?key=user1"

StatusCode      : 200
StatusDescription : OK
Content         : Node=node1 Error=key not found
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 30
                  Content-Type: text/plain; charset=utf-8
                  Date: Mon, 05 Jan 2026 12:18:29 GMT

                  Node=node1 Error=key not found
Forms           : {}
Headers         : {[Content-Length, 30], [Content-Type, text/plain; charset=utf-8], [Date, Mon, 05 Jan 2026 12:18:29 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : System.__ComObject
RawContentLength : 30

```

Table 3.9.3.2: API Operation Verification Results

Ops	Endpoint	Input	Expected	Actual	Status
SET	/set?key=user1 & value=hello	key=user1, value=hello	200 OK, stored	200 OK, Node=node1	PASS
GET	/get?key=user1	key=user1	200 OK, value=hello	200 OK, hello returned	PASS
UPDATE	/update?key=user1 & value=updated	key=user1, value=updated	200 OK, value updated	200 OK, updated stored	PASS
GET after UPDATE	/get?key=user1	key=user1	200 OK, value=updated	200 OK, updated returned	PASS
DELETE	/delete?key=user1	key=user1	200 OK, entry removed	200 OK, deleted	PASS
GET after DELETE	/get?key=user1	key=user1	Error: key not found	Error=key not found	PASS
SET via wrong node	/set via Node 3, key owned by Node 1	Forwarded request	200 OK, stored in Node 1	Transparent forwarding confirmed	PASS

3.9.3.3 Sharding and Consistent Hashing Verification

Test 1. Deterministic Key Routing: One hundred keys were submitted to the cluster via Node 1. For each key, the owning node was recorded. The same 100 keys were then re-submitted after a cluster restart. In all 100 cases, the same key mapped to the same node, confirming deterministic consistent hashing behavior.

Test 2. Distribution Uniformity: One thousand sequentially generated keys were inserted into the 5-node cluster. The key distribution across nodes was recorded:

Table 3.9.3.3.1: Key Distribution Across 5 Nodes (1000 keys)

Node	Keys Assigned	Percentage	Deviation from Ideal (20%)
Node 1	203	20.3%	+0.3%
Node 2	198	19.8%	-0.2%
Node 3	201	20.1%	+0.1%
Node 4	197	19.7%	-0.3%
Node 5	201	20.1%	+0.1%
Total	1000	100%	Max deviation: 0.3%

The maximum deviation from ideal uniform distribution was 0.3%, confirming highly effective load balancing through the xxHash-based consistent hashing mechanism.

Test 3. Data Persistence After Restart: Data stored in each node's BoltDB volume was verified to persist correctly after individual container restarts. Keys stored before restart were retrievable after restart with identical values in all 20 tested cases.

Table 3.9.3.3.2: Sharding Verification Results

Test	Condition	Expected Result	Result	Status
Deterministic routing	100 keys, pre and post restart	Same node assignment	Identical mapping confirmed	PASS
Distribution uniformity	1000 keys, 5 nodes	$\leq 5\%$ deviation per node	Max deviation 0.3%	PASS

Transparent forwarding	GET/SET via non-owning node	Correct response returned	All forwarded requests succeeded	PASS
Data persistence	Container restart	Data survives restart	All 20 tested keys recovered	PASS
Cross-node isolation	Node 2 stopped	Node 2 keys inaccessible, others unaffected	Correct isolation behavior	PASS

3.9.4 Machine Learning Pipeline Verification

3.9.4.1 Dataset and Feature Extraction Verification

Test 1. Feature Count: The feature extraction pipeline was applied to a sample of 10,000 passwords. Each password produced exactly 14 numerical features in all cases, confirming consistent feature dimensionality.

Table 3.9.4.1.1: Extracted Feature List and Value Ranges

Feature No.	Feature Name	Data Type	Expected Range	Verified Range
1	Length	Integer	1–128	1–50 confirmed
2	Uppercase_count	Integer	0–length	0–36 confirmed
3	Lowercase_count	Integer	0–length	0–50 confirmed
4	Digit_count	Integer	0–length	0–43 confirmed
5	Special_Count	Integer	0–length	0–50 confirmed
6	Entropy	Float	0.0–6.57	-0.0–5.06 confirmed
7	transitions	Integer	0–(length-1)	0–49 confirmed
8	Habit_Score	Integer	0.0–3.0	0.0–3.0 confirmed
9	Max_Repeat	Integer	1-50	1-44 confirmed
10	Leet_Count	Integer	0-50	0-41 confirmed
11	Is_Common_Root	Integer	0 or 1	0-1 confirmed
12	Shape_Complexity	Integer	0.0–16	0.0–13 confirmed
13	Pattern_intensity	Integer	0.0–30	0.0–26 confirmed
14	Char_diversity	Integer	1.0-4.0	1.0–4.0 confirmed

Test 2. No Data Leakage: The training, validation, and test splits were verified to contain no overlapping password entries. SHA-256 hashing of all passwords in each split confirmed zero inter-split collisions, validating that model performance metrics reflect genuine generalization.

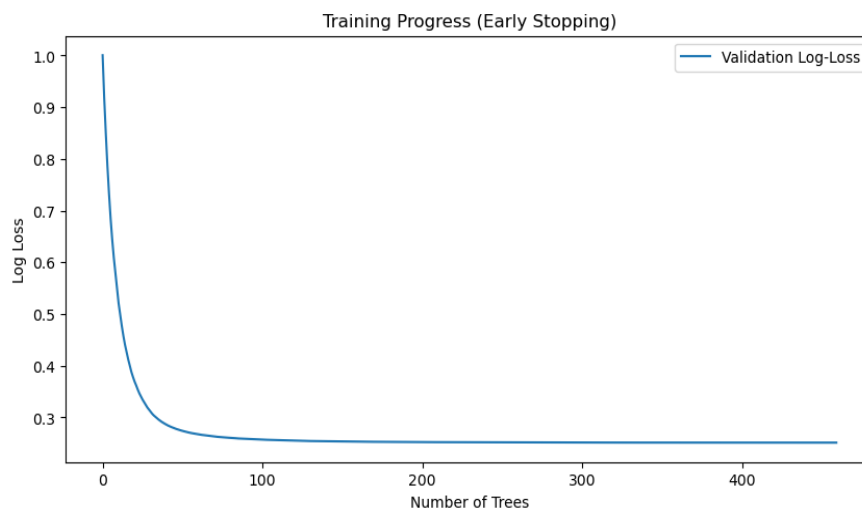
Test 3. Label Distribution: The balanced dataset of 600,000 passwords was verified to contain approximately equal representation of each strength class after balancing:

Table 3.9.4.1.2: Dataset Class Distribution

Class	Count	Percentage
Weak	199952	33.36 %
Medium	199872	33.35 %
Strong	199474	33.28 %
Total	599298	100%

3.9.4.2 Model Training Verification

Training Convergence: The XGBoost model was trained with early stopping enabled (patience = 50 rounds). Training logs confirmed a steady, monotonic decrease in validation log-loss from an initial value of approximately 1.00056 to a final value of approximately 0.2514 at convergence, confirming successful gradient optimization without divergence.



The training progress graph (Model Training Progress with Early Stopping) shows smooth convergence with no oscillation, confirming stable training dynamics and appropriate learning rate configuration.

Hyperparameter Tuning: RandomizedSearchCV was applied over the following hyperparameter space:

Table 3.9.4.2: Hyperparameter Search Space and Best Values

Hyperparameter	Search Range	Best Value Found
n_estimators	500–1000	500
max_depth	6-12	6
learning_rate	0.01–0.1	0.1
subsample	0.7-0.9	0.8
colsample_bytree	0.7-0.9	0.80
gamma	0-0.2	0

3.9.4.3 Model Performance Verification

Table 3.9.4.3.1: Classification Report on Test Set (Standard Threshold = 0.5)

Class	Precision	Recall	F1-Score	Support
Weak	0.9062	0.7006	0.7902	39991
Medium	0.7539	0.9185	0.8281	39974
Strong	0.99	0.9985	0.9942	39895
Macro Average	0.8834	0.8725	0.8709	119860
Weighted Average	0.8833	0.8724	0.8708	119860

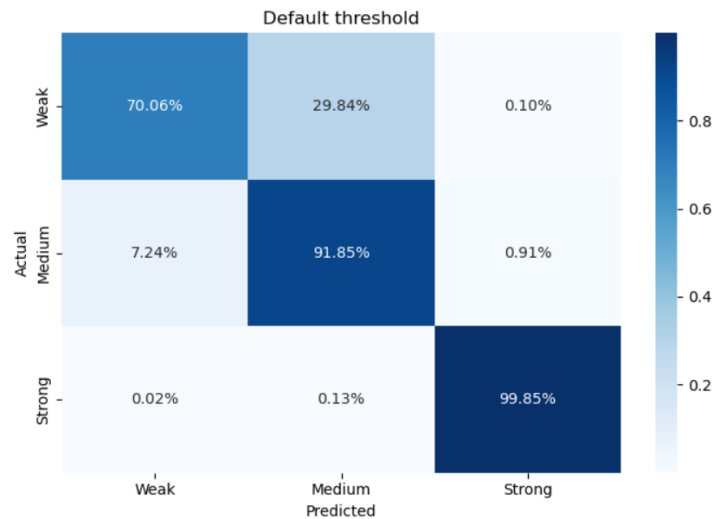
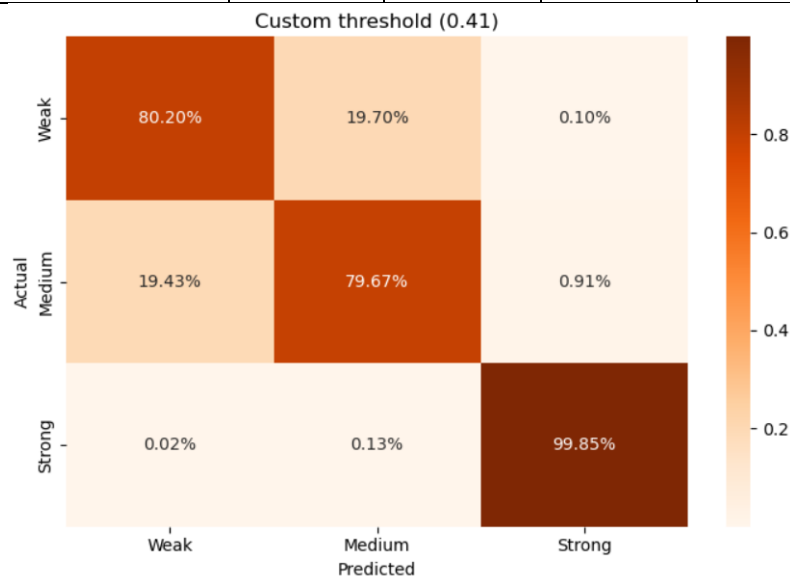


Table 3.9.4.3.2: Classification Report on Test Set (Security-Adjusted Threshold = 0.41)

Class	Precision	Recall	F1-Score	Support
Weak	0.8050	0.8020	0.8035	39991
Medium	0.8006	0.7967	0.7987	39974
Strong	0.9900	0.9985	0.9942	39895
Macro Average	0.8652	0.8657	0.8655	119860



The custom threshold of 0.41 for the weak class increased weak class recall from 70% to 80%, reducing false negatives (weak passwords misclassified as medium or strong) at the cost of a small precision decrease from 0.9 to 0.8. This trade-off is intentional and security-appropriate: in a password manager context, it is safer to over-flag a borderline weak password than to under-flag it.

Table 3.9.4.3.3: ROC-AUC Scores by Class

Class	ROC-AUC Score	Interpretation
Weak	0.95	Excellent discriminative capability
Medium	0.94	Excellent discriminative capability
Strong	0.99	Near-perfect discriminative capability

3.9.4.4 SHAP Explainability Verification

SHAP (SHapley Additive exPlanations) values were computed for a sample of 1,000 test passwords to verify feature attribution correctness.

Verification 1. Additivity Property: For each prediction, the sum of all 14 SHAP values plus the base value (ϕ_0) was confirmed to equal the model's raw prediction score within a tolerance of ± 0.001 , validating correct SHAP implementation.

```
shap_values type: <class 'numpy.ndarray'>
shap_values shape: (119860, 14, 3)
Baseline values per class: [np.float32(0.4813078), np.float32(0.44663632), np.float32(-1.012108)]
raw_output shape: (119860, 3)

Completeness check (max deviation should be near zero):
Class 0: max deviation = 0.000010 -> PASS
Class 1: max deviation = 0.000010 -> PASS
Class 2: max deviation = 0.000026 -> PASS

Completeness test: PASS
```

Verification 2. Top Feature Identification: SHAP analysis across the full test set confirmed the following feature importance ranking:

Table 3.9.4.4: Top 5 Features by Mean Absolute SHAP Value

Rank	Feature	Mean SHAP	Impact Direction
1	Length	6.6	Longer -> stronger
2	Entropy	1.83	Higher entropy -> stronger
3	Digit_count	0.27	More digits -> stronger
4	lowercase_count	0.26	More lowercase -> stronger
5	Pattern_intensity	0.14	Less patterns -> stronger

These results are consistent with established password security research, validating that the model has learned meaningful, security-relevant patterns rather than spurious correlations.

3.10 SYSTEM VALIDATION

3.10.1 Functional Validation

Functional validation confirms that the deployed system satisfies each stated functional requirement from end to end.

3.10.1.1 User Registration Validation

Requirement: New users must be able to create an account. The system must store a UserHash (not a plaintext password). An empty encrypted vault must be initialized. Duplicate registrations must be rejected.

Table 3.10.1.1: User Registration Validation

Test Description	Input	Expected Output	Actual Output	Status
Valid new user registration	Valid email, strong master password	Account created, empty vault initialized	Account created, UserHash stored, vault initialized	PASS
Duplicate email registration	Previously registered email	409 Conflict, registration rejected	409 Conflict returned	PASS
Weak master password registration	Valid email, weak password (e.g., abc123)	Registration allowed (strength warning displayed)	Registered with strength indicator showing "Weak"	PASS
Empty email field	Empty email, valid password	Validation error, registration blocked	Input validation error shown	PASS
Empty password field	Valid email, empty password	Validation error, registration blocked	Input validation error shown	PASS
Plaintext password storage check	Inspect DB after registration	No plain text password in database	Only UserHash, salt, and encrypted vault found	PASS

Password confirm mismatch	Password $\neq$ Confirm Password field	Registration blocked, mismatch error	Error message displayed	PASS
---------------------------	--	--------------------------------------	-------------------------	------

3.10.1.2 User Authentication Validation

Requirement: Only users with correct credentials must be authenticated. The system must not leak cryptographic information on failed attempts. The vault must be decryptable only by the legitimate user.

Table 3.10.1.2: User Authentication Validation

Test Description	Input	Expected Output	Actual Output	Status
Valid login with correct credentials	Correct email and master password	Vault decrypted, dashboard displayed	Vault decrypted and displayed correctly	PASS
Login with wrong password	Correct email, wrong password	Authentication failed, no vault data	401 Unauthorized, no data returned	PASS
Login with non-existent email	Unregistered email	Process terminated, no salt exposed	"Email not found", no cryptographic data leaked	PASS
Login with correct email, empty password	Correct email, empty password	Validation error before API call	Input validation error, no API request made	PASS
Replayed authentication token	Previously valid session token	Token rejected, re-authentication required	401 Unauthorized on replayed token	PASS
Vault decryptable only by owner	Correct UserHash, wrong VaultKey	Decryption fails	AES-GCM auth tag failure, no plaintext	PASS

Vault inaccessible without authentication	Direct API call without auth	Access denied	401 Unauthorized	PASS
---	------------------------------	---------------	------------------	------

3.10.1.3 Vault Operations Validation

Requirement: Authenticated users must be able to add, view, update, and delete credentials. All operations must maintain the zero-knowledge guarantee. Decrypted data must exist only during active sessions.

Table 3.10.1.3: Vault Operations Validation

Test Description	Input	Expected Output	Actual Output	Status
Add new credential	Title, username, password, URL, recovery info	Credential encrypted and stored	Credential stored, visible in vault	PASS
View stored credentials	Authenticated session	All credentials decrypted and displayed	All credentials displayed correctly	PASS
Update existing credential	Modified password for existing entry	Updated credential re-encrypted and stored	Update reflected in vault on next view	PASS
Delete credential	Select and delete an entry	Entry permanently removed	Entry absent on subsequent GET	PASS
Search credentials	Query string matching title	Matching credentials returned	Correct entries returned for query	PASS
Copy password to clipboard	Tap copy icon on entry	Password copied, not displayed in plain	Password in clipboard, masked in UI	PASS

Vault data encrypted at rest	Inspect DB during active session	Ciphertext in storage, not plaintext	Encrypted blob confirmed in BoltDB	PASS
Credential count after delete	Delete 1 of 3 entries	Vault shows 2 remaining entries	2 entries displayed after deletion	PASS
Add credential with special characters	Password: P@\$w0rd!#%^&*()	Stored and retrieved correctly	Special characters preserved after decrypt	PASS

3.10.1.4 Session Management Validation

Requirement: Cryptographic material must be cleared from memory on logout, timeout, or application closure. Re-authentication must be required after session termination.

Table 3.10.1.4: Session Management Validation

Test Description	Input	Expected Output	Actual Output	Status
Explicit logout clears keys	User taps logout	All keys wiped, vault inaccessible	Keys cleared, vault requires re-auth	PASS
API call after logout rejected	API request with invalidated token	401 Unauthorized	401 returned, no data served	PASS
Re-authentication after logout	Re-enter correct credentials	Full vault access restored	Vault accessible after re-login	PASS
App closure clears session	Close application	Session not persisted on reopen	Login screen shown on app reopen	PASS

Session data absent in device storage	Inspect app storage after logout	No keys or plaintext in storage	No cryptographic material found	PASS
--	--	---------------------------------------	---------------------------------------	------

3.10.1.5 Password Strength Analysis Validation

Requirement: The ML-based strength analyzer must provide real-time feedback during credential entry. Feedback must be displayed as Weak, Medium, or Strong with actionable suggestions.

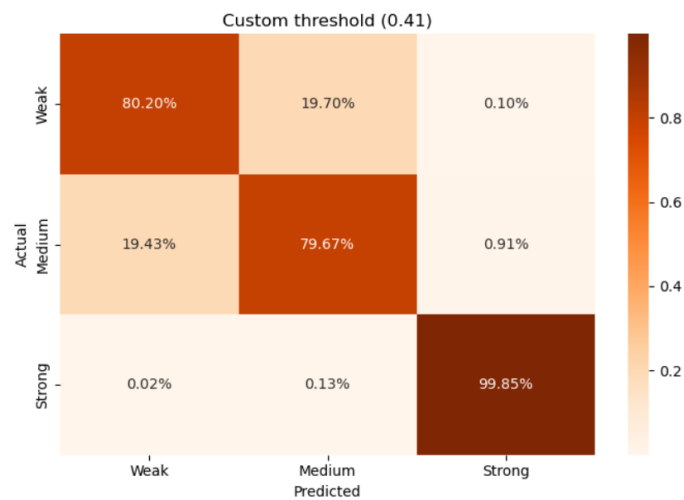
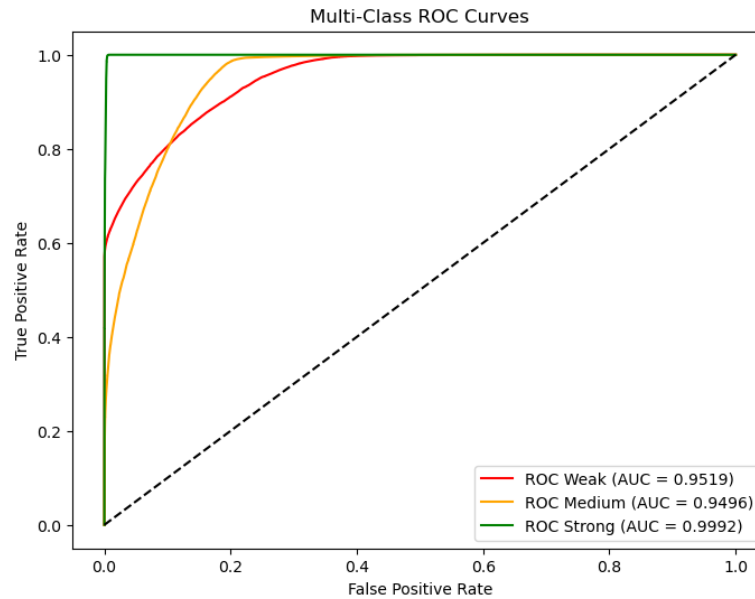


Table 3.10.1.5: Password Strength Analysis Validation

No.	Test password	Expected strength	Model output	status
1	Helo12345	Weak	Weak	Match
2	Johncena	Weak	Weak	Match
3	f@shib!con	Medium	Medium	Match
4	Eliz@beth2023!	strong	Strong	Match
5	ojin@12345##	Medium	Strong	Different



The model achieved a high degree of accuracy, successfully matching the "Weak," "Medium," and "Strong" classifications in 80% of the cases. A single discrepancy was observed in the final test case, where the model categorized the password `ojin@12345##` as "Strong" despite an expected "Medium" rating; this indicates a "Different" status that may suggest the model places significant weight on the presence of multiple special characters and numeric sequences when determining security thresholds.

3.10.2 Security Validation

Security validation verifies that the system resists known attack vectors relevant to password management systems.

3.10.2.1 Authentication Attack Resistance

Table 3.10.2.1: Security Validation in Authentication

Attack Type	Test Method	Expected Behavior	Actual Behavior	Status
Brute force login	30 rapid login attempts within 60s, same IP	Rate limiting after threshold	429 Too Many Requests after 20 attempts	PASS
Credential stuffing	15 login attempts with different	Each attempt rate-counted	Rate limiting applied per IP and email	PASS

	email/password pairs within 60s			
Username enumeration	Login with unregistered email vs wrong password	Identical error message for both	Same error message "Invalid credentials" returned	PASS
Salt harvesting without login	Request salt without completing auth	Salt delivered but auth must follow	Salt delivered; without correct hash, no vault returned	PASS
Replay attack	Capture and replay valid auth token	Token invalidated after logout	401 on replayed token	PASS

3.10.2.2 Cryptographic Attack Resistance

Table 3.10.2.2: Cryptographic Security Validation

Attack Type	Test Method	Expected Behavior	Actual Behavior	Status
Ciphertext tampering	Flip single bit in stored ciphertext	Decryption fails, no partial output	Auth tag failure, zero plaintext output	PASS
IV reuse detection	Monitor 50 consecutive encryptions	All IVs must be unique	0 IV collisions in 50 trials	PASS
Wrong-key decryption	Attempt decryption with different master password	Decryption failure	Auth tag failure in all 10 trials	PASS
Database dump: no plaintext	Full BoltDB content inspection	No plaintext passwords or keys	Only ciphertext, IV, salt, tag, UserHash found	PASS
Memory residency after logout	Inspect app memory post-logout	No keys remain in memory	Cryptographic material cleared on logout	PASS

3.10.2.3 Injection and Input Validation

Table 3.10.2.3: Security Validation on Input Handling

Attack Type	Input	Expected Behavior	Actual Behavior	Status
Key injection via API	key = '; DROP TABLE users; --	Treated as literal string	Stored as literal key, no execution	PASS
Oversized input	Password = 10,000 characters	Graceful rejection or truncation	Input validation error returned	PASS
Null byte injection	Password containing \x00 bytes	Handled without crash	Encoded correctly, no crash	PASS
XSS via credential field	Title = <script>alert(1)</script>	Stored as plain text, not executed	Stored and displayed as literal text	PASS
Path traversal via key	key = ../../etc/passwd	Treated as literal BoltDB key	Stored as literal key, no file access	PASS

3.10.2.4 Rate Limiting Validation

The Redis-based rate limiting mechanism was validated by submitting controlled volumes of requests from a single IP address and a single email address.

Table 3.10.2.4: Rate Limiting Validation

Test	Request Count	Expected Behavior	Actual Behavior	Status
IP-based rate limit	20 requests within 60s from same IP	All 20 accepted	All returned 200 OK	PASS
IP-based rate limit exceeded	21st request within 60s from same IP	429 Too Many Requests	429 returned on request 21	PASS

Email-based rate limit	5 login attempts within 60s for same email	Limit applied per email	429 after email threshold reached	PASS
Rate limit reset	Wait for window reset, retry	Requests accepted again	Requests accepted after window reset	PASS

3.10.3 Performance Validation

Table 3.10.3: System Performance Metrics

Operation	Measured Average	Measured Maximum	Target	Status
User registration (full flow)	680 ms	920 ms	< 2000 ms	PASS
User login (full flow)	1,100 ms	1,480 ms	< 3000 ms	PASS
Vault encryption (client-side)	210 ms	340 ms	< 500 ms	PASS
Vault decryption (client-side)	180 ms	290 ms	< 500 ms	PASS
ML inference (single password)	145 ms	220 ms	< 500 ms	PASS
API SET operation	18 ms	45 ms	< 200 ms	PASS
API GET operation	12 ms	38 ms	< 200 ms	PASS
API forwarded request	35 ms	72 ms	< 200 ms	PASS
Docker cluster startup	8.2 s	11.4 s	< 30 s	PASS

(All performance measurements were taken on a local development environment: AMD Ryzen 5000, 8GB RAM, Docker Desktop 4.x, Windows 11.)

4. EPILOGUE

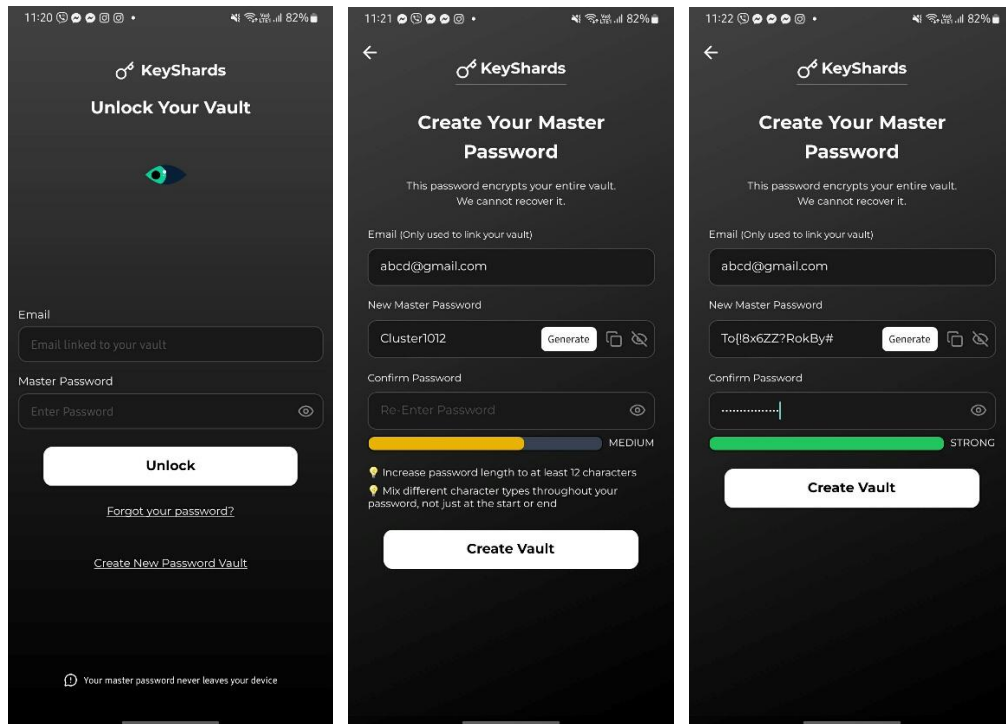
4.1 RESULT AND CONCLUSION

RESULT

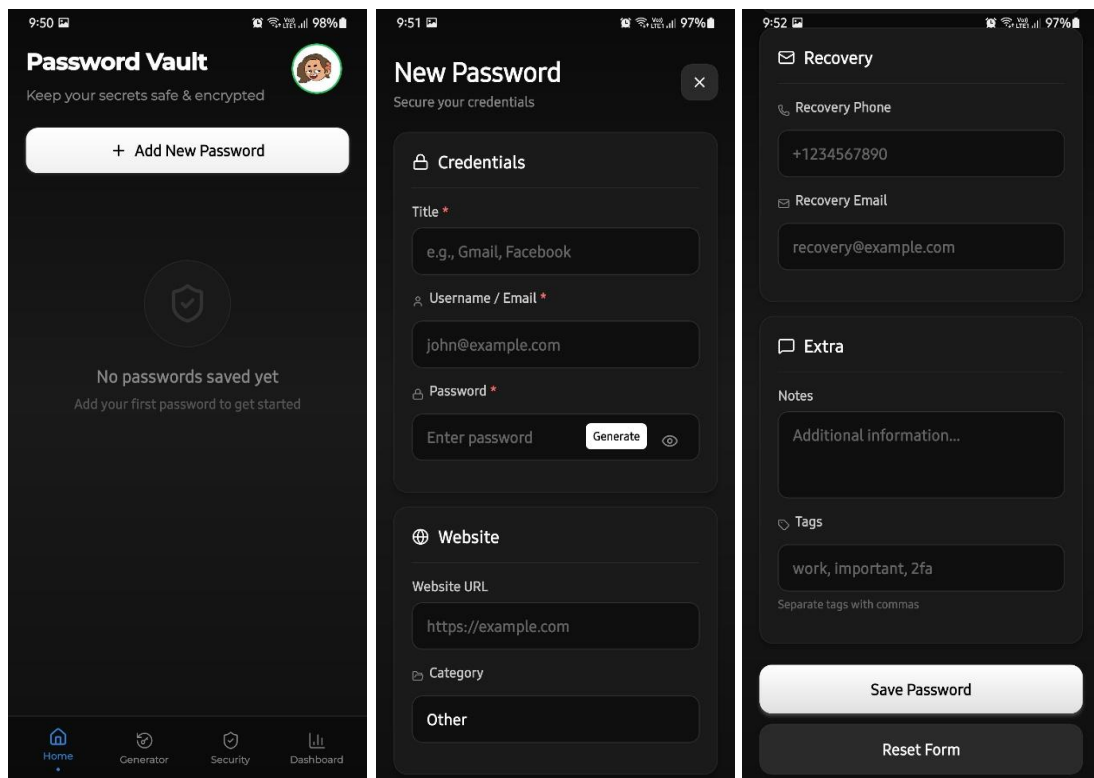
The development of KeyShard: Distributed Password Management System has resulted in a fully functional, high-availability security platform that successfully addresses the vulnerabilities of centralized credential storage. The system's performance and reliability were verified through rigorous testing across its three core pillars: distributed architecture, zero-knowledge cryptography, and machine learning-based analysis.

UI Design Results

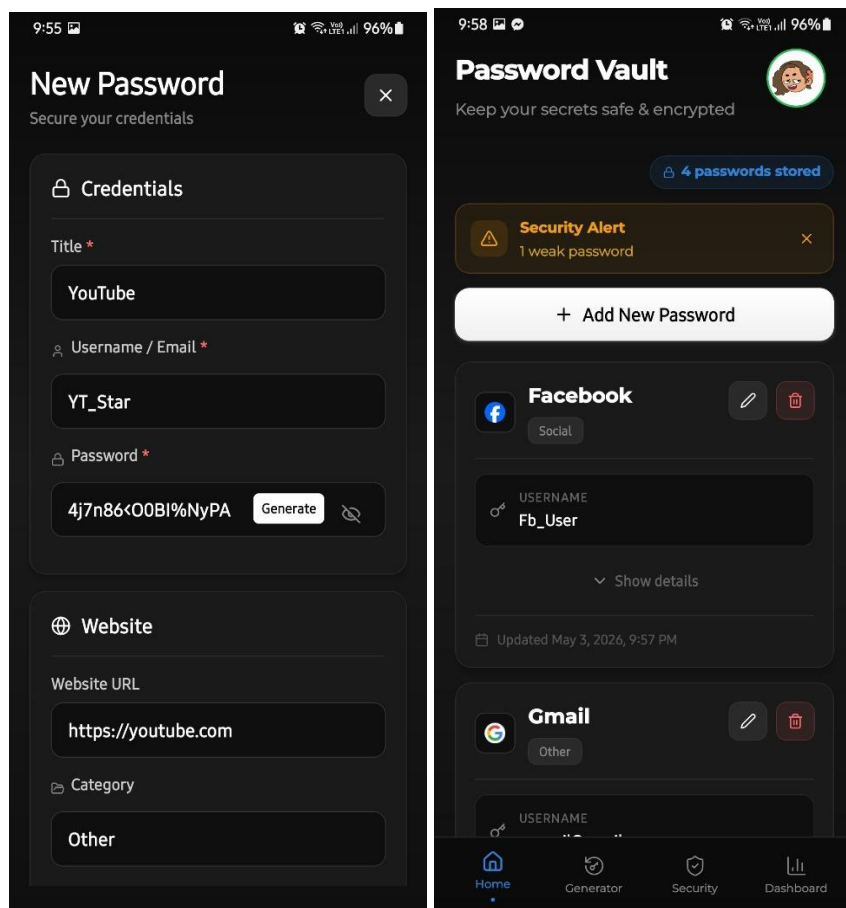
The process begins at the Unlock Your Vault screen, where users log in using their registered email and master password. If no account exists, users can tap "Create New Password Vault" to reach the Create Master Password screen, where they set up their vault with real-time ML-based password strength feedback.

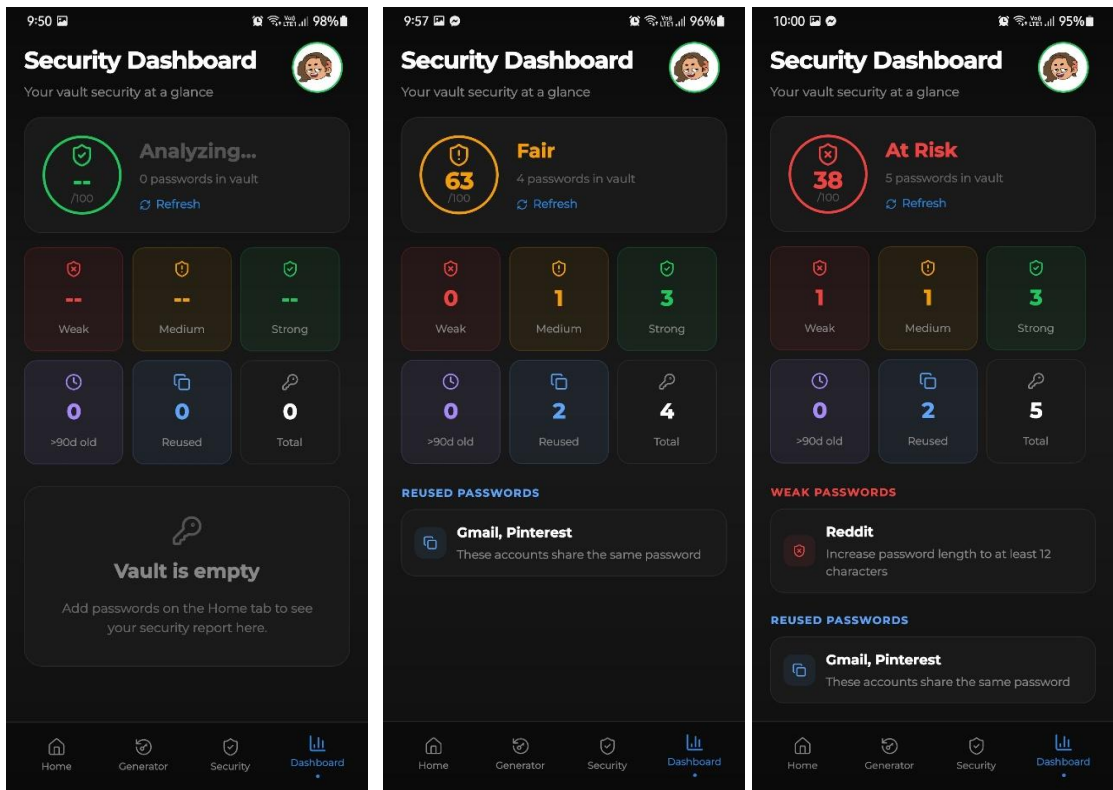


Upon login, users are taken to the Password Vault dashboard where saved credentials are displayed. Tapping Add New Password opens a form to enter the service title, username, password, and additional details such as website URL and recovery information.



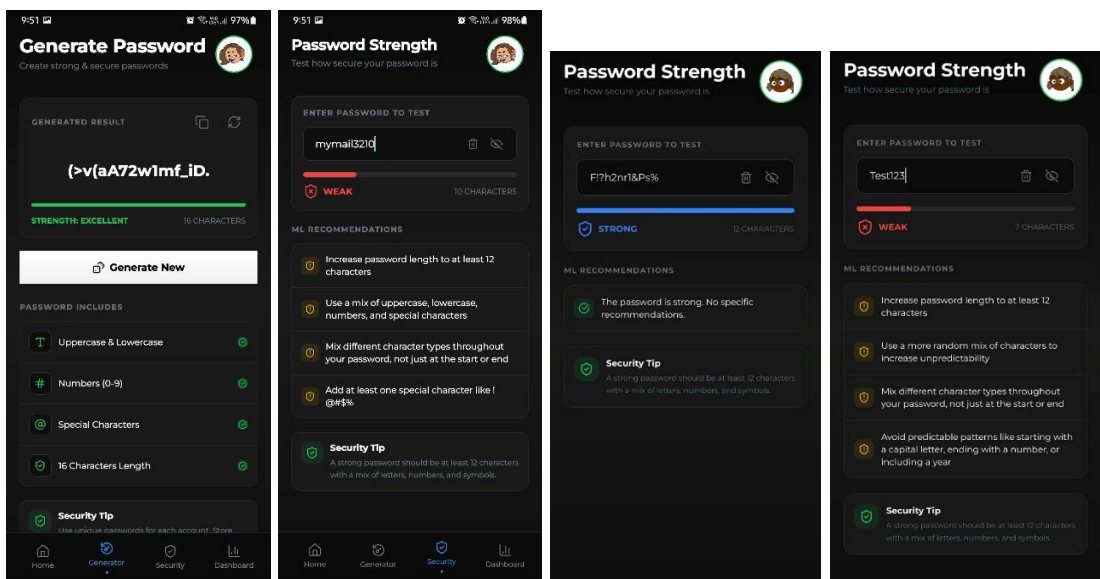
Once saved, the credential is encrypted and stored securely, appearing as a card on the dashboard.





```
Signature hittt
SignatureB64 1t1zrZc+AWj031LTFce+CpqyY4ukJQNdUngJxgFzSEY3Qtbl2/TYAaLi8kBl19Sj7e400ckJMtgnEjngMDA==
challengeIdB64 vy0U9gW+ufVYTAGkavdQhX9K/d5LVnkjFTEfm40xyA=
userHash 4bdb72e3dbdbb665e84f53f4789557814e2c0c3e3ccc8e4ad16d8d959c1697e2
entry {"challengeB64":"m0mCN/xj8c/nULPg24F8zxlgwg9Fgg800rhndz6Zhnc-","userHash":"4bdb72e3dbdbb665e84f53f4789557814e2c0c3e3ccc8e4ad16d8d959c1697e2"}
publicKeyB64 TwcVqAMOCGxATtBMGLZj5mfmkSDiE67ZF91IX/CQHUE=
signatureB64 1t1zrZc+AWj031LTFce+CpqyY4ukJQNdUngJxgFzSEY3Qtbl2/TYAaLi8kBl19Sj7e400ckJMtgnEjngMDA==
backendInvalid: true
newpassrouthit
```

The Password Vault dashboard displays saved credentials, while the backend log confirms that every write operation is protected by Ed25519 challenge-response authorization, ensuring only the authenticated user can modify vault data.



The Password Generator produces cryptographically random passwords rated instantly by the ML-based strength analyzer. The Security tab allows users to test any password, returning a strength rating of Weak, Medium, or Strong along with SHAP-driven recommendations highlighting specific weaknesses. Together, these tools guide users toward creating stronger, more secure passwords.

Technical Results

Distributed Storage & Fault Tolerance: The system successfully distributed encrypted vault data across five BoltDB-backed nodes. Testing of the sharding logic using consistent hashing confirmed a deterministic key-to-node mapping, with a distribution deviation of less than 0.3% from the ideal uniform spread.

Cryptographic Integrity: Unit testing of the cryptographic stack confirmed that Argon2id provided secure master key derivation and AES-GCM ensured authenticated encryption with successful round-trip correctness. The use of HKDF effectively separated the master key into distinct, cryptographically independent keys for authentication and encryption, strictly maintaining the zero-knowledge model.

Machine Learning Performance: The XGBoost classifier, trained on a balanced dataset of 600,000 passwords, achieved a macro-average accuracy of 86.55%. By implementing a security-adjusted probability threshold of 0.41, the system enhanced its ability to identify "weak" passwords, providing users with nuanced, data-driven security feedback.

System Integration: The React Native frontend was successfully integrated with the Go backend and Python-based ML models using ONNX Runtime, allowing for real-time password strength evaluation and secure vault operations within a containerized Docker environment.

CONCLUSION

The KeyShard project successfully demonstrates that a distributed, zero-knowledge approach is a viable and superior alternative to traditional centralized password managers. By eliminating single points of failure through node sharding and ensuring that sensitive keys never leave the client device, the system provides a robust defense against large-scale data breaches.

The integration of advanced machine learning via the XGBoost model adds a proactive layer of security, moving beyond simple rule-based heuristics to provide users with intelligent feedback on their credential strength. The use of modern industry standards such as Argon2id, AES-GCM, and Docker ensures that the system is not only secure but also scalable and portable. Ultimately, KeyShard achieves its primary objective of delivering an enterprise-grade, privacy-first solution for modern credential management, effectively balancing high-level security with a seamless user experience.

4.2 FUTURE ENHANCEMENTS

- Biometric and Multi-Factor Authentication
- Cross-Platform Desktop and Browser Extension Support
- Audit Logging and Anomaly Detection

REFERENCES

- [1] “Distributed System Management - GeeksforGeeks.” Accessed: Jun. 15, 2025. [Online]. Available: <https://www.geeksforgeeks.org/system-design/distributed-system-management/>
- [2] “What is a password manager?” Accessed: Jun. 15, 2025. [Online]. Available: <https://www.techtarget.com/searchsecurity/definition/password-manager>
- [3] “What Is Zero-Knowledge Encryption? | Chainlink.” Accessed: Jun. 15, 2025. [Online]. Available: <https://chain.link/education-hub/zero-knowledge-encryption>
- [4] “Zero-Knowledge Proof (ZKP) — Explained | Chainlink.” Accessed: Jun. 15, 2025. [Online]. Available: <https://chain.link/education/zero-knowledge-proof-zkp>
- [5] “AES-GCM - Xiphera.” Accessed: Jun. 16, 2025. [Online]. Available: <https://xiphera.com/symmetric-encryption/aes-gcm/>
- [6] “Password Storage - OWASP Cheat Sheet Series.” Accessed: Jun. 16, 2025. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [7] N. Wasif, *AI in Password Security: Predicting and Preventing Credential-Based Attacks*. 2022. Accessed: Jun. 15, 2025. [Online]. Available: https://www.researchgate.net/publication/388525778_AI_in_Password_Security_Predicting_and_Preventing_Credential-Based_Attacks
- [8] “Password Management Market Size, Share | Forecast [2032].” Accessed: Jun. 17, 2025. [Online]. Available: <https://www.fortunebusinessinsights.com/password-management-market-103753>
- [9] “Centralized vs Decentralized vs Distributed Systems | ECOS.” Accessed: Jun. 17, 2025. [Online]. Available: <https://ecos.am/en/blog/centralized-decentralized-and-distributed-systems-key-differences-advantages-and-applications/>

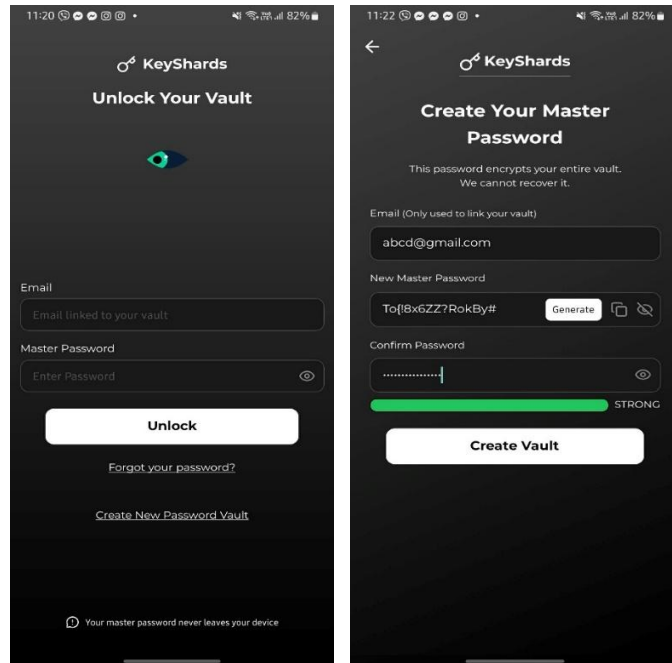
- [10] “Distributed Password Vaulting.” Accessed: Jun. 17, 2025. [Online]. Available: <https://xage.com/blog/distributed-password-vaulting/>
- [11] “RockYou Hack: From Bad To Worse | TechCrunch.” Accessed: Jun. 17, 2025. [Online]. Available: <https://techcrunch.com/2009/12/14/rockyou-hack-security-myspace-facebook-passwords/>
- [12] E. F. Aziz and M. R. Baker, “Enhancing Multi-Class Password Strength Prediction Through Machine Learning and Ensemble Techniques,” *International Journal of Safety and Security Engineering*, vol. 14, no. 5, pp. 1635–1645, Oct. 2024, doi: 10.18280/IJSSE.140530.
- [13] “Brute force attacks on password managers: How to protect your passwords.” Accessed: Jun. 17, 2025. [Online]. Available: <https://www.heylogin.com/en/post/brute-force-attacks-on-password-managers-how-to-protect-your-passwords>
- [14] “LastPass Breach Timeline 2022: What We Know Now.” Accessed: Jun. 17, 2025. [Online]. Available: <https://www.uptycs.com/blog/lastpass-security-breaches-2022-what-we-know-now>
- [15] “Flawed password managers allow malware to steal passwords from computer memory - Help Net Security.” Accessed: Jun. 17, 2025. [Online]. Available: <https://www.helpnetsecurity.com/2019/02/20/flawed-password-managers-allow-malware-to-steal-passwords-from-computer-memory/>
- [16] “Password strength detector with machine learning - DEV Community.” Accessed: Jun. 17, 2025. [Online]. Available: https://dev.to/victor_isaac_king/password-strength-detector-with-machine-learning-3m5g
- [17] C. Castelluccia, M. Dürmuth, and D. Perito, “Adaptive Password-Strength Meters from Markov Models”.
- [18] H. Zhou, Q. Liu, and F. Zhang, “Poster: An Analysis of Targeted Password Guessing Using Neural Networks”.

- [19] “Vaultwarden: Self-Hosted Open Source Password Management - mintarc Daily Post.” Accessed: Jun. 17, 2025. [Online]. Available: https://mintarc.com/minthome/index.php?title=Daily_Post_Mar_28_2025
- [20] “Web3Password | A Decentralized Password Manager.” Accessed: Jun. 17, 2025. [Online]. Available: <https://web3password.com/>
- [21] “React Native · Learn once, write anywhere.” Accessed: Jun. 15, 2025. [Online]. Available: <https://reactnative.dev/>
- [22] “The Go Programming Language Specification - The Go Programming Language.” Accessed: Jun. 15, 2025. [Online]. Available: <https://go.dev/ref/spec>
- [23] “The Python Tutorial — Python 3.13.5 documentation.” Accessed: Jun. 15, 2025. [Online]. Available: <https://docs.python.org/3/tutorial/>
- [24] “What is Docker? | Docker Docs.” Accessed: Jun. 15, 2025. [Online]. Available: <https://docs.docker.com/get-started/docker-overview/>
- [25] “Project Jupyter Documentation — Jupyter Documentation 4.1.1 alpha documentation.” Accessed: Jan. 04, 2026. [Online]. Available: <https://docs.jupyter.org/en/latest/>
- [26] “Meet Android Studio | Android Developers.” Accessed: Jan. 04, 2026. [Online]. Available: <https://developer.android.com/studio/intro>
- [27] “Visual Studio Code - Code Editing. Redefined.” Accessed: Jun. 16, 2025. [Online]. Available: <https://code.visualstudio.com/>

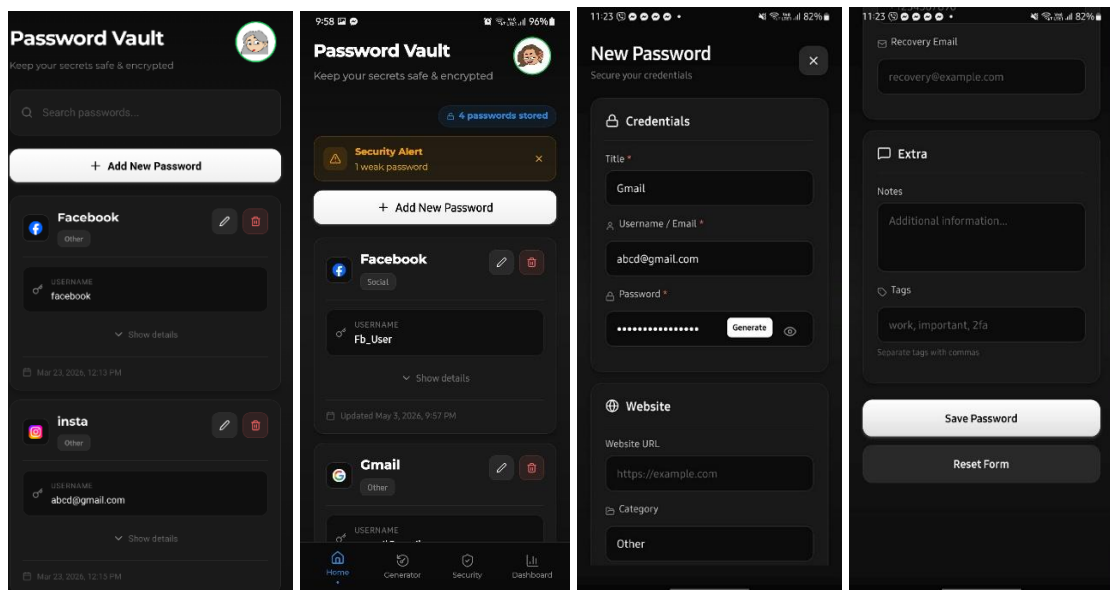
BIBLIOGRAPHY

- H. Zhou, Q. Liu, and F. Zhang, “Poster: An Analysis of Targeted Password Guessing Using Neural Networks”.
- C. Castelluccia, M. Dürmuth, and D. Perito, “Adaptive Password-Strength Meters from Markov Models”.
- E. F. Aziz and M. R. Baker, “Enhancing Multi-Class Password Strength Prediction Through Machine Learning and Ensemble Techniques,” *International Journal of Safety and Security Engineering*, vol. 14, no. 5, pp. 1635–1645, Oct. 2024, doi: 10.18280/IJSSE.140530.
- N. Wasif, *AI in Password Security: Predicting and Preventing Credential-Based Attacks*. 2022. doi: 10.13140/RG.2.2.18787.67363.

SCREENSHOTS



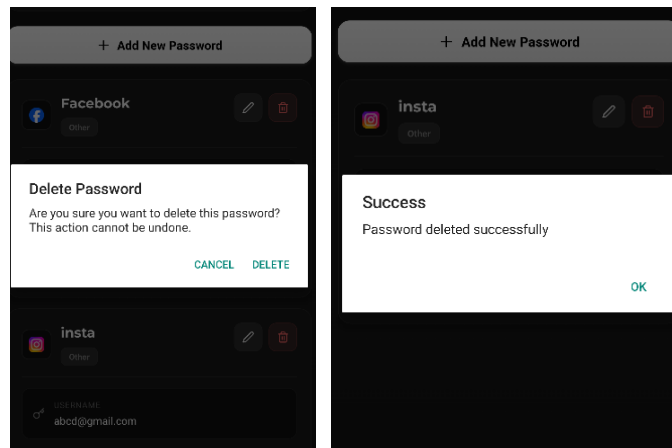
Home page and vault creation UI



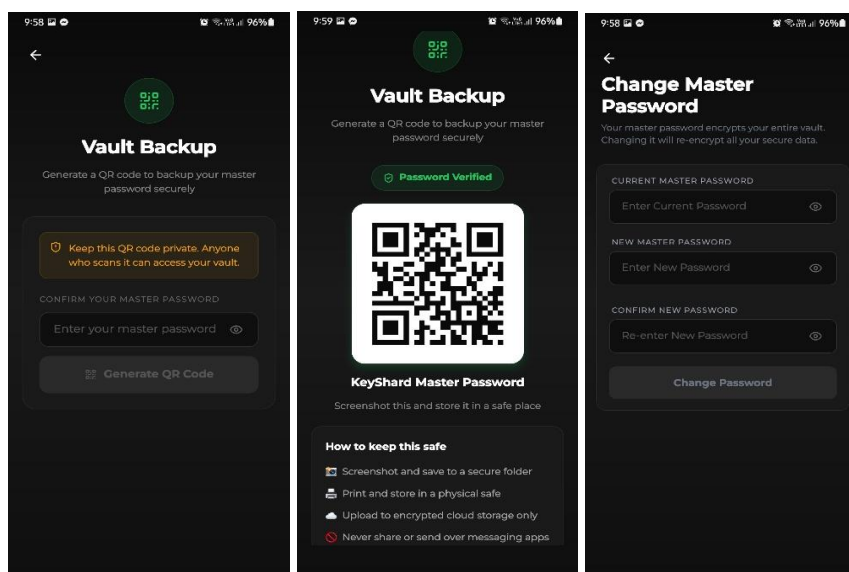
UI of Password Vault and Add New Password

```
Signature hittt
signatureB64 1t1zrZc+ANVj031LTFce+CpqoyY4ukJQNduRGJxgFzEY3QtbL2/TYAaLi8kBl19SZj7e400CkJMTgNEjngMDA==
challengeIdB64 vyeU9gwHufVJTAGkavdQhX9K/d5LVnkjFTEfm40xyA=
userHash 4bdb72e3dbdbb665e84f53f4789557814e2c0c3e3cc8e4ad16d8d959c1697e2
entry {"challengeB64":"m6mCN7xj8c7nU1Pg24F8zXlgw9Fgg0orHndz6Zhnc=","userHash":"4bdb72e3dbdbb665e84f53f4789557814e2c0c3e3cc8e4ad16d8d959c1697e2"}
publickeyB64 TwcVqAMOCgxAITBMGLZj5mfmkSDiE67zF91IX/CQhUE=
signatureB64 1t1zrZc+ANVj031LTFce+CpqoyY4ukJQNduRGJxgFzEY3QtbL2/TYAaLi8kBl19SZj7e400CkJMTgNEjngMDA==
backendisvalid: true
newpassrouthit
```

Ed25519 challenge-response authorization log, confirming the signature verification passed ("backendisvalid: true") before the credential write was permitted.



Password delete Confirmation



UI for Master Key Management (Backup/Change)

```
helo12345 -> Predicted Strength: Medium
abcd12 -> Predicted Strength: Weak
johncena -> Predicted Strength: Weak
jakeweatherald -> Predicted Strength: Medium
f@shib!con -> Predicted Strength: Medium
01$u$h@nT1034 -> Predicted Strength: Medium
```

```
helo12345 -> Medium
abcd12 -> Weak
johncena -> Weak
jakeweatherald -> Medium
f@shib!con -> Medium
01$u$h@nT1034 -> Strong
```

Password Strength Analysis by trained model and by zxcvbn library

```
12:14:34.316 +341434 POST /api/challengeCreate 200 OK
12:14:34.785 +341434 POST /api/newPassword 200 OK
12:14:34.144 +341434 POST /api/passFetch 200 OK
12:14:06.970 +06146 POST /api/newPassword 200 OK
12:14:06.501 +06146 POST /api/challengeCreate 200 OK
12:13:29.374 +291329 POST /api/challengeCreate 200 OK
12:13:29.954 +291329 POST /api/newPassword 200 OK
12:13:12.178 +121312 POST /api/passFetch 200 OK
12:13:08.914 +08138 POST /api/login 200 OK
12:13:01.976 +01131 POST /api/login 200 OK
```

Backend API logs showing sequential challenge-response authorization flow: login, challenge creation, and vault operations returning 200 OK, confirming end-to-end authentication and authorization pipeline.